

Doca

Manual técnico do projeto

Pedidos de venda, compras e estoque em Python e FastAPI

Feito por Erik L. Pimenta

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

Visão geral 4

- Para que serve 4
- O que o Doca faz 4
- Tecnologias 5
- Identidade visual 5

Como rodar 6

- Primeira execução 6
- Passo a passo manual 6
- Com Docker (PostgreSQL + API) 7
- Estrutura de pastas 7

Configuração 8

- Variáveis de ambiente 8
- Gerar a chave secreta 8
- Regras de produção 9
- Conferências 9

Acesso e perfis 10

- Contas de demonstração 10
- Regras da sessão 11
- Perfis e permissões 12

Guia do painel 13

- Painel 13
- Pedidos 13
- Estoque 17
- Compras 18
- Produtos 20
- Clientes e fornecedores 21
- Relatórios 23

Usuários e auditoria 25
Tema escuro 27
No celular 28
Pedidos, estoque e compras 29
Ciclo do pedido 29
Reserva sem venda dupla 30
Regras de valor 30
Tipos de movimentação 31
Ciclo da compra 31
Arquitetura 32
Camadas do código 32
Decisões principais 32
Erros 33
API 34
Exemplo de uso 35
Banco de dados 36
Tabelas 37
Migrações 37
Usar PostgreSQL fora do Docker 37
Segurança e privacidade 38
Testes, CI e automação 39
Testes 39
Integração contínua 39
Relatório diário automático 40
Dados fictícios e publicação 41
Dados fictícios 41
Publicar o seu fork sem expor dados pessoais 41
Solução de problemas 42

Capítulo 1

Visão geral

O Doca é um sistema completo de pedidos de venda, compras e controle de estoque em vários depósitos. Ele cobre o caminho inteiro de uma venda: o pedido nasce como rascunho, reserva o estoque ao ser confirmado, passa pela separação, faturamento e envio, e termina na entrega. Tudo isso com uma API REST, um painel web com login e relatórios gerenciais.

Para que serve

O projeto foi feito para ser usado, estudado e adaptado. Qualquer pessoa pode baixar o código, rodar com dois cliques e entrar com as contas de demonstração para ver seis meses de operação fictícia funcionando. Para usar de verdade, basta trocar o banco, definir as próprias senhas no arquivo `.env` e cadastrar os produtos reais.

O que o Doca faz

Operação

- **Pedidos de venda** com rascunho editável, confirmação com reserva de estoque, separação, faturamento, envio com código de rastreio, entrega e cancelamento com liberação da reserva ou estorno.
- **Estoque por depósito** com saldo físico, reservado e disponível, entradas, ajuste por contagem (inventário), transferências entre depósitos e um livro de movimentações que nunca é apagado.
- **Compras** ao fornecedor com envio, recebimento total ou parcial e recálculo do custo médio ponderado.

Cadastros e gestão

- Produtos com SKU, EAN-13, categoria, fornecedor, preço, custo e estoque mínimo.
- Clientes pessoa física e jurídica com validação de CPF e CNPJ, limite de crédito e histórico de compras.
- Fornecedores com prazo de entrega e depósitos com cidade e UF.
- Relatórios de vendas por dia, semana, mês, categoria, produto, vendedor, cliente ou UF, curva ABC, giro, cobertura em dias e sugestão de compra, todos exportáveis em CSV.
- Usuários com quatro perfis, senha provisória com troca obrigatória e trilha de auditoria.

Por baixo

- Reserva atômica com trava no banco: nenhum pedido vende acima do disponível, mesmo com acessos simultâneos.
- Dinheiro guardado como inteiro de centavos, sem erro de arredondamento.
- Máquina de estados declarativa: a mesma tabela valida as ações no servidor e decide quais botões aparecem na tela.
- Funciona com SQLite (padrão, sem instalar nada) ou PostgreSQL, com migrações versionadas pelo Alembic.

Tecnologias

Camada	Tecnologia	Papel
Linguagem	Python 3.12 ou mais recente	Toda a regra de negócio e a API
API	FastAPI, Pydantic v2, Uvicorn	Rotas REST, validação de entrada e documentação interativa
Banco	SQLAlchemy 2 (ORM tipado), Alembic	Mapeamento das tabelas e migrações
Bancos suportados	SQLite e PostgreSQL 17	SQLite para uso local, PostgreSQL no Docker e em produção
Autenticação	JWT (PyJWT), scrypt	Sessões revogáveis e senhas com sal individual
Interface	HTML, CSS e JavaScript puros (ES modules)	Painel sem etapa de build e sem CDN
Fonte	Archivo (servida localmente)	Tipografia de toda a interface
Testes	Pytest, Ruff	80 testes, lint e formatação
Entrega	Docker, Docker Compose, GitHub Actions	Imagem sem privilégios, healthchecks e integração contínua

Identidade visual

A interface é sóbria, pensada para a operação diária e para tabelas densas: fundo cinza-claro, painéis brancos com bordas finas e o azul-marinho como único destaque (item ativo do menu, gráficos, abas e foco).

#F3F4F6	**#FFFFFF**	**#E1E4E8**	**#5B626B**	**#16191D**	**#2B4C7E**
Fundo das páginas	Painéis, tabelas e formulários	Bordas e divisórias	Textos de apoio	Texto e botões principais	Destaque e foco

As cores de situação foram mantidas, mais sóbrias, só nos marcadores das etapas:

#8A9099	**#3B64B0**	**#6D5BB5**	**#2A7F86**	**#B07A2A**	**#2F7D4F**	**#B0463F**
Rascunho	Confirmado	Separado	Faturado	Enviado	Entregue	Cancelado

Tema escuro: fundo `#0F1113`, painéis `#16191C`, texto `#E6E8EB` e destaque `#7FA2DB`.

Tipografia. Toda a interface usa a Archivo em largura normal, servida pelo próprio projeto (`web/fonts/`), com números tabulares nas tabelas.

Componentes. Cantos de 4 a 6 px; tabelas com cabeçalho em cinza suave; links sem sublinhado (ele aparece ao passar o mouse); filtros em fichas retangulares, a ativa em azul-claro; selects com seta própria; avisos escuros no canto inferior direito, os de erro em vermelho.

A marca é o monograma Do. Ela aparece em vetor no menu lateral, na tela de entrada e no ícone da aba (`web/img/icone.svg`). As versões em imagem ficam em `docs/img/logo/`: fundo preto, fundo branco e transparente.

Capítulo 2

Como rodar

O Doca roda em qualquer computador com Python 3.12 ou mais recente. Sem configurar nada, ele usa um banco SQLite local já preenchido com dados fictícios.

Primeira execução

Windows

1. Instale o Python 3.12 ou mais recente.
2. Baixe ou clone o repositório.
3. Dê dois cliques em `iniciar.bat`.
4. Na primeira vez o script copia `.env.example` para `.env`, cria o ambiente virtual, instala as dependências e gera seis meses de operação fictícia. As senhas de demonstração aparecem na janela e ficam salvas em `dados/credenciais_demo.txt`.
5. O navegador abre em http://127.0.0.1:8000. Para parar, feche a janela do terminal.

Linux ou macOS

```
./iniciar.sh
```

Passo a passo manual

```
cp .env.example .env              # Windows (PowerShell): Copy-Item .env.example .env
python -m venv .venv
.venv\Scripts\activate            # Windows (Linux/macOS: source .venv/bin/activate)
pip install -r requirements-dev.txt
python -m scripts.popular_banco   # dados fictícios (use --limpar para recomeçar)
uvicorn app.main:app --reload --host 127.0.0.1
```

O gerador aceita parâmetros para mudar o volume de dados:

Parâmetro	Padrão	Função
`--meses`	6	Meses de operação simulados
`--pedidos`	700	Pedidos de venda gerados
`--clientes`	160	Clientes fictícios
`--produtos`	80	Produtos no catálogo
`--semente`	42	Mesma semente gera os mesmos dados
`--limpar`	—	Apaga todas as tabelas antes de popular

Com Docker (PostgreSQL + API)

1. Preencha no `.env` as variáveis `CHAVE_SECRETA`, `POSTGRES_BANCO`, `POSTGRES_USUARIO` e `POSTGRES_SENHA` (veja o capítulo Configuração).
2. Suba os serviços:

```
docker compose up -d --build
docker compose logs api        # mostra as credenciais de demonstração
```

3. Acesse http://127.0.0.1:8000 (ou a porta definida em `PORTA_API`).

Se alguma variável obrigatória estiver vazia, o Docker para e avisa qual falta. O `docker/entrypoint.sh` aplica as migrações antes de subir a API, e as portas ficam presas a `127.0.0.1`: só o próprio computador acessa.

Estrutura de pastas

Pasta	Conteúdo
`app/main.py`	Criação do app, middlewares, rotas e arquivos estáticos
`app/config.py`	Configurações lidas do `.env`
`app/database.py`	Engine, sessão e tipos (dinheiro exato, data e hora UTC)
`app/seguranca.py`	scrypt, JWT e limitador de tentativas de login
`app/middleware.py`	Cabeçalhos de segurança, CSRF e limite de requisições
`app/modelos/`	Tabelas SQLAlchemy e enums
`app/esquemas/`	Contratos de entrada e saída (Pydantic)
`app/servicos/`	Regras de negócio: estoque, pedidos, compras, relatórios e auditoria
`app/rotas/`	Endpoints da API
`web/`	Painel: HTML, CSS, JavaScript, fonte e ícone
`alembic/`	Migrações do banco
`scripts/`	Gerador de dados fictícios e relatório diário
`tests/`	Testes de unidade, API e concorrência
`docs/`	Capturas de tela, logo, guia de publicação, relatório diário e este manual
`dados/`	Criada na primeira execução: banco local, chave e senhas geradas (fora do Git)

Capítulo 3

Configuração

Todas as opções ficam no arquivo `.env`, que fica só no seu computador e nunca vai para o Git. O modelo é o `.env.example`.

Variáveis de ambiente

Variável	Padrão	Função
`AMBIENTE`	`dev`	`dev`, `teste` ou `producao`
`DATABASE_URL`	SQLite em `dados/`	Outro banco, por exemplo `postgresql://<usuario>:<senha>@<servidor>:5432/<banco>`
`CHAVE_SECRETA`	gerada em `dados/`	Assinatura dos tokens. Obrigatória no Docker e em produção
`TOKEN_MINUTOS`	30	Validade da sessão, em minutos
`COOKIE_SEGURO`	`false`	`true` quando o sistema estiver atrás de HTTPS
`ADMIN_USUARIO`	`admin`	Login do primeiro administrador
`ADMIN_SENHA`	aleatória	Senha do primeiro administrador; vazia grava uma em `dados/senha_inicial_admin.txt`
`SENHA_DEMO`	aleatória	Senha dos usuários de demonstração; vazia grava em `dados/credenciais_demo.txt`
`CRIAR_TABELAS`	`true`	Cria as tabelas sozinho ao subir (desligado no Docker, que usa migrações)
`DOCS_PUBLICAS`	`true`	Mostra a documentação interativa em `/docs`
`LOGIN_MAX_TENTATIVAS`	5	Tentativas erradas antes do bloqueio
`LIMITE_REQUISICOES_MINUTO`	300	Limite geral de requisições por minuto
`POSTGRES_BANCO`, `POSTGRES_USUARIO`, `POSTGRES_SENHA`	—	Banco criado pelo Docker Compose
`PORTA_API` / `PORTA_BANCO`	8000 / 5432	Portas publicadas no seu computador

Gerar a chave secreta

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Copie o resultado para `CHAVE_SECRETA`. Em uso local ela pode ficar vazia: o sistema gera uma e guarda em `dados/.chave_secreta`.

Regras de produção

Com `AMBIENTE=producao`, o Doca se recusa a subir se a `CHAVE_SECRETA` tiver menos de 32 caracteres ou se `COOKIE_SEGURO` não estiver em `true`. Assim, uma instalação real não vai ao ar com chave fraca ou com o cookie de sessão trafegando sem HTTPS.

Conferências

- O `.env` está no `.gitignore`. Rode `git status` e confirme que ele não aparece.
- A pasta `dados/`, com banco local, chave e senhas geradas, também é ignorada pelo Git.

Capítulo 4

Acesso e perfis

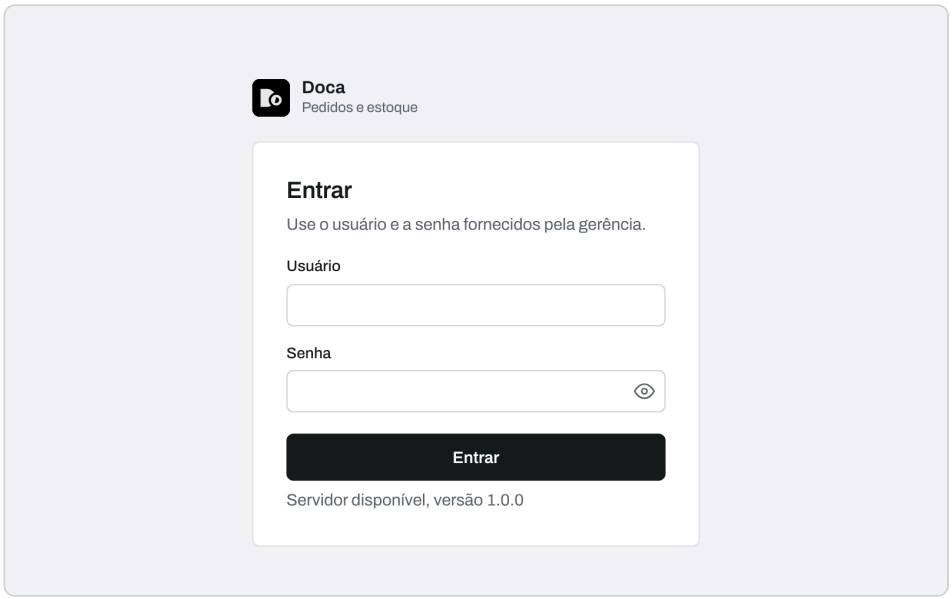
Toda tela e toda rota da API exigem login. O painel guarda a sessão num cookie protegido e as integrações usam o mesmo token no cabeçalho `Authorization: Bearer`.

Contas de demonstração

Usuário	Perfil	Uso típico
`admin`	Administração	Tudo, incluindo usuários e auditoria
`gerente.demo`	Gerência	Pedidos, estoque, compras, cadastros e relatórios
`vendas1.demo`, `vendas2.demo`, `vendas3.demo`	Vendas	Os próprios pedidos e consulta de estoque
`estoque.demo`	Estoque	Separação, faturamento, envio, entrega e movimentação de estoque

O repositório não traz senhas. Na primeira execução elas são geradas e gravadas em `dados/credenciais_demo.txt` e `dados/senha_inicial_admin.txt`, que ficam só no seu computador. Para escolher as suas, preencha `ADMIN_SENHA` e `SENHA_DEMO` no `.env` antes de gerar os dados.

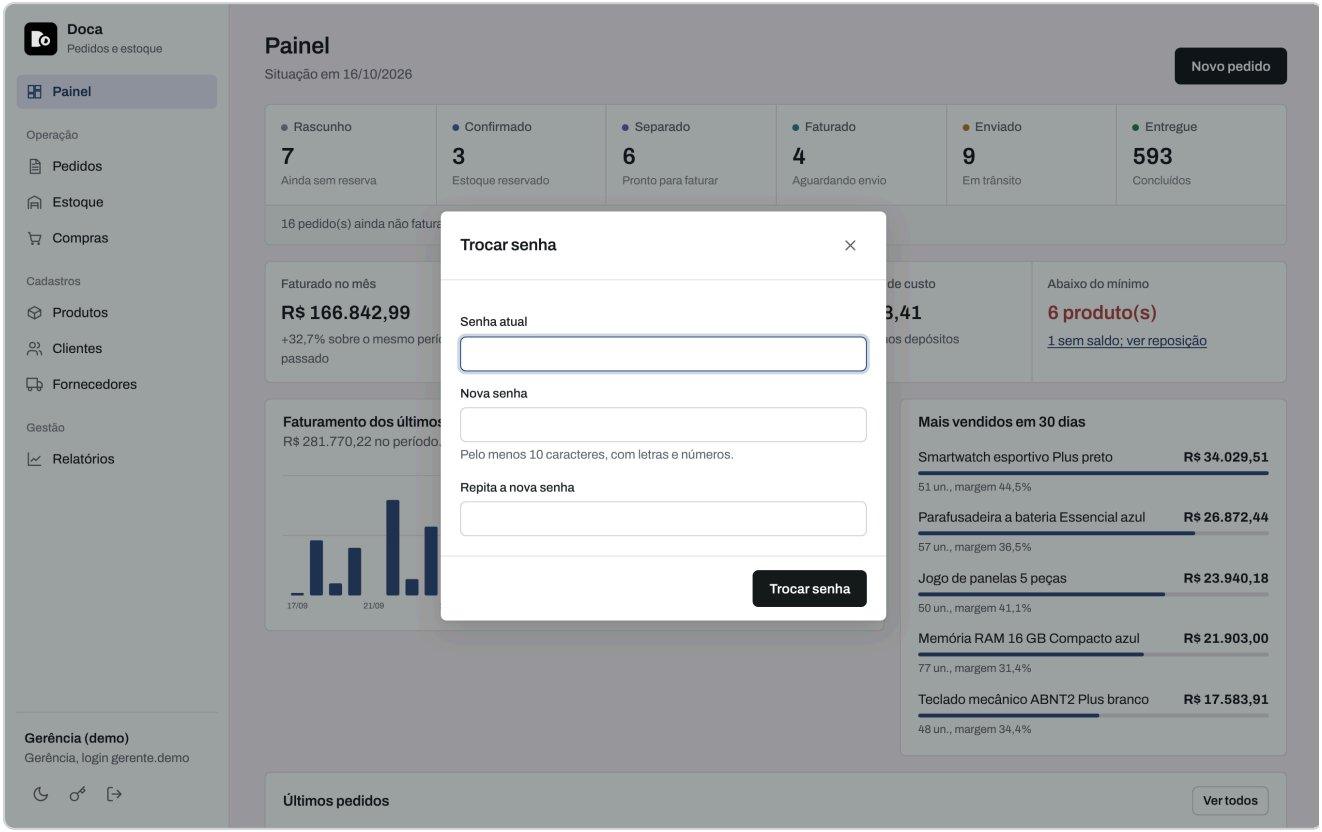
A tela de entrada mostra a marca com o nome **Doca** e a linha "Pedidos e estoque" acima de um cartão branco **Entrar**, com os campos de usuário e senha (com botão para mostrar a senha digitada) e, abaixo do botão, o status do servidor com a versão em uso.



Tela de entrada

Regras da sessão

Regra	Como funciona
Validade	O token vale 30 minutos (`TOKEN_MINUTOS`). Depois disso o painel volta para a tela de entrada.
Cookie do painel	`HttpOnly` e `SameSite=Strict`: o JavaScript da página não lê o token e outro site não consegue usá-lo.
Bloqueio	5 tentativas erradas em 10 minutos bloqueiam novas tentativas por 5 minutos, por origem e por usuário.
Resposta igual	Usuário inexistente e senha errada recebem a mesma mensagem, no mesmo tempo.
Senha provisória	Usuários criados ou com senha redefinida pelo administrador precisam trocar a senha no primeiro acesso.
Força da senha	Pelo menos 10 caracteres, letras e números, sem conter o nome de usuário.
Revogação	Trocar a senha, desativar o usuário, mudar o perfil ou usar *sair de todos* encerra todas as sessões abertas.



Troca de senha, pelo ícone de chave no rodapé do menu

Perfis e permissões

Ação	Administração	Gerência	Vendas	Estoque
Criar, editar e confirmar pedidos	sim	sim	os seus	não
Separar, faturar, enviar e entregar	sim	sim	não	sim
Cancelar pedido faturado (estorno)	sim	sim	não	não
Entrada, ajuste e transferência de estoque	sim	sim	não	sim
Criar compras	sim	sim	não	não
Receber mercadoria	sim	sim	não	sim
Cadastrar produtos e preços	sim	sim	não	não
Relatórios de vendas e curva ABC	sim	sim	não	não
Reposição e giro	sim	sim	sim	sim
Ver CPF completo do cliente	sim	sim	revelar (auditado)	não
Usuários e auditoria	sim	não	não	não

Desconto máximo por item: vendas até 10%, gerência até 30%, administração sem limite.

Capítulo 5

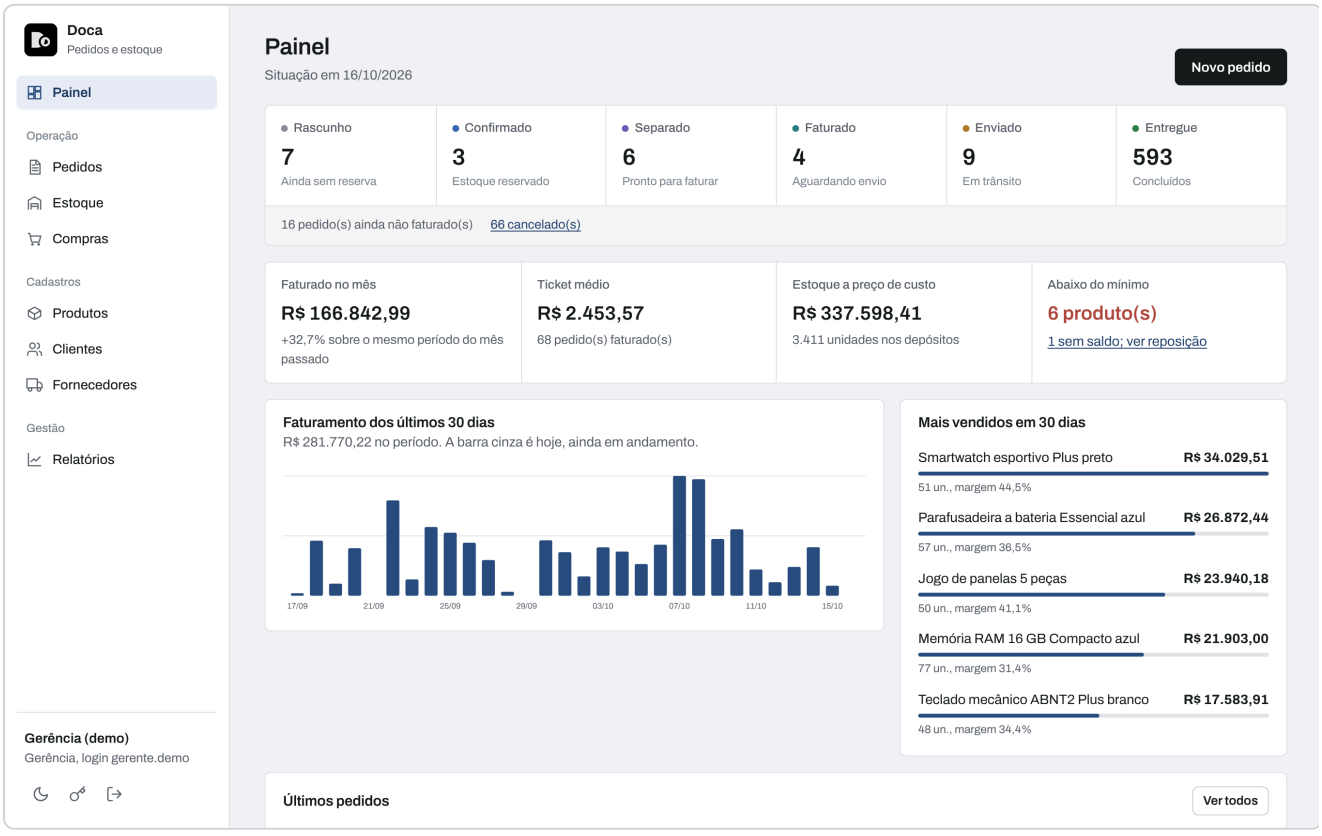
Guia do painel

O painel tem um menu lateral dividido em Operação, Cadastros e Gestão; o item da tela aberta fica com fundo azul-claro e texto azul. Os filtros de cada lista ficam no endereço da página, então dá para guardar ou compartilhar um link já filtrado. No rodapé do menu ficam o nome do usuário, a troca de tema, a troca de senha e o botão de sair.

Painel

A primeira tela tem o título **Painel** e a data da situação ("Situação em dd/mm/aaaa"). Logo abaixo, os pedidos por etapa ficam numa faixa única com seis colunas (rascunho, confirmado, separado, faturado, enviado e entregue), cada uma com o marcador da cor da situação, a quantidade e uma dica curta, como "Ainda sem reserva" ou "Pronto para faturar". O rodapé da faixa mostra quantos pedidos ainda não foram faturados e traz o link para os cancelados.

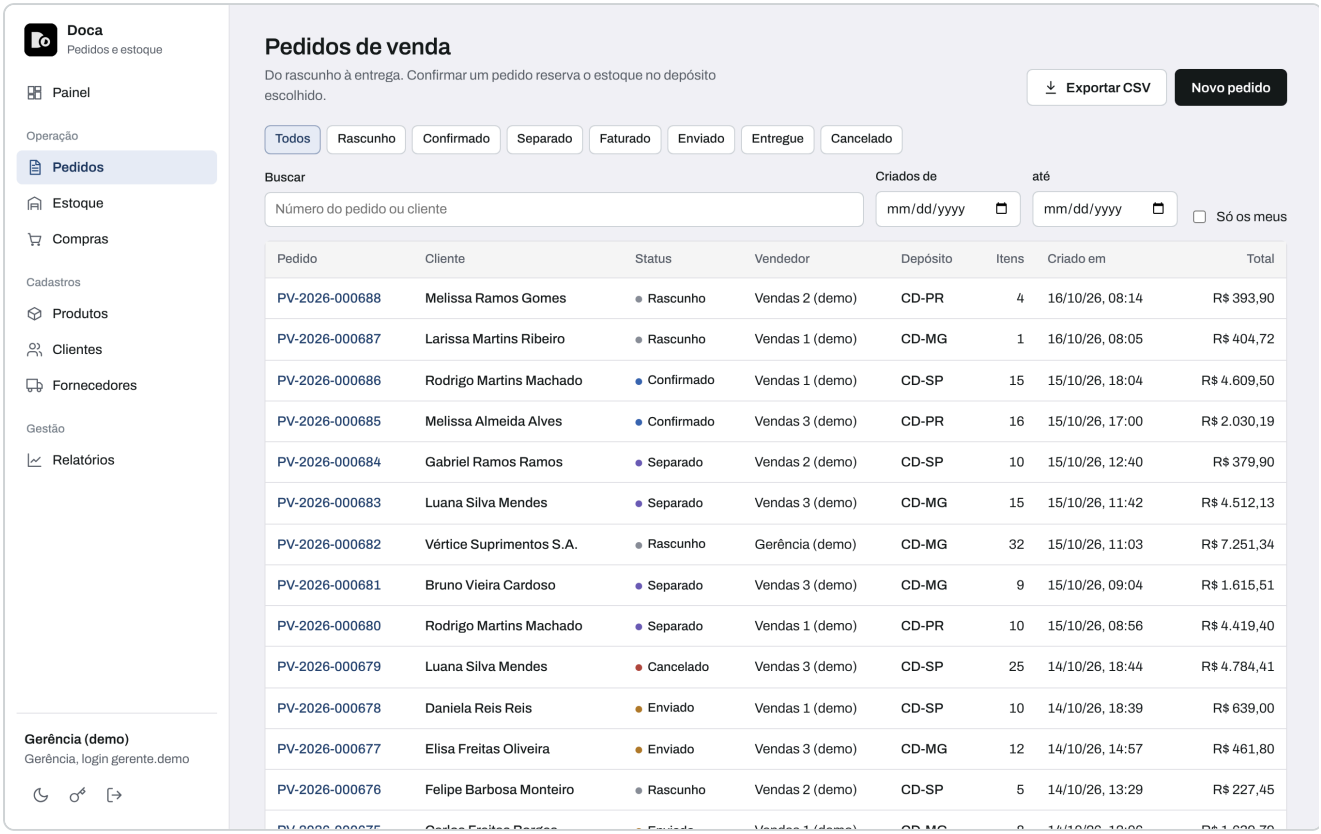
Em seguida vem a faixa de indicadores do mês: o faturamento comparado ao mesmo período do mês anterior, o ticket médio, o estoque a preço de custo e os produtos abaixo do mínimo, com atalho para a sugestão de reposição. Abaixo vêm o gráfico de faturamento dos últimos 30 dias, com as barras em azul-marinho (a barra de hoje fica cinza, porque o dia ainda está em andamento), os mais vendidos com margem e um trilho fino proporcional à receita, e os últimos pedidos.



Painel inicial

Pedidos

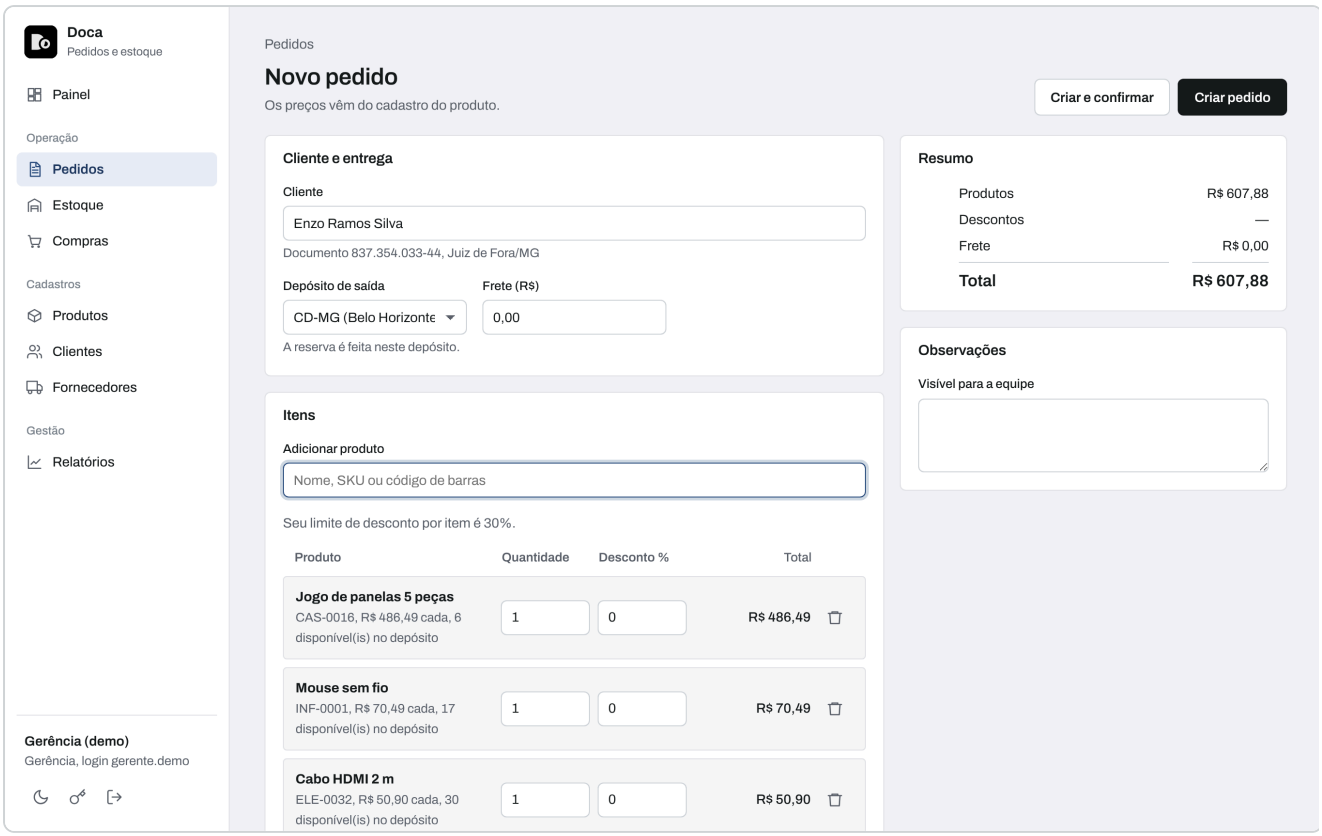
A lista de pedidos de venda tem abas por situação, busca por número ou cliente, filtro por depósito, vendedor e período, e exportação em CSV. Vendedores veem só os próprios pedidos.



Lista de pedidos de venda

Novo pedido

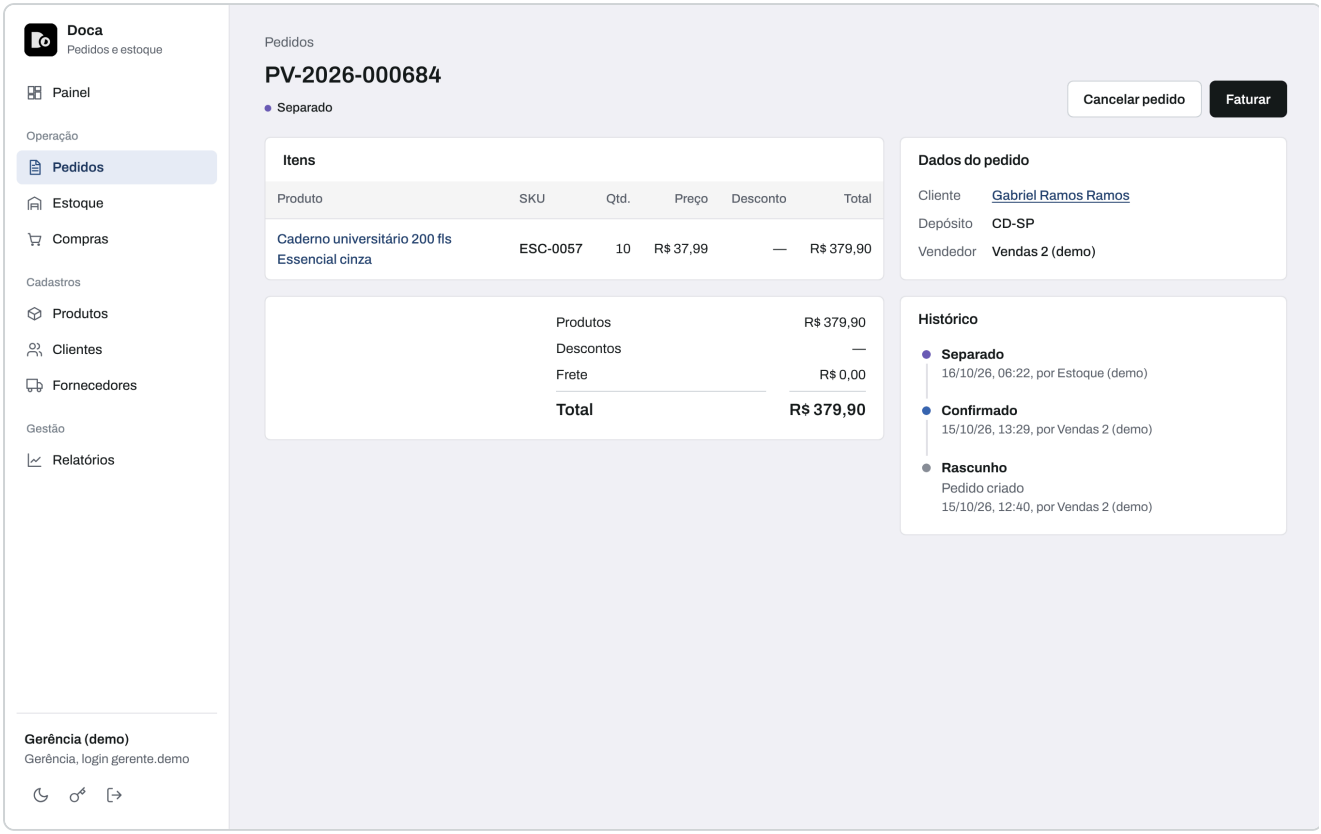
Escolha o cliente (por nome, cidade ou documento), o depósito de saída e o frete. Os produtos são buscados por nome, SKU ou código de barras; cada item mostra o preço de cadastro e o disponível no depósito escolhido. O resumo recalcula produtos, descontos, frete e total a cada alteração. **Criar pedido** salva como rascunho; **Criar e confirmar** já reserva o estoque.



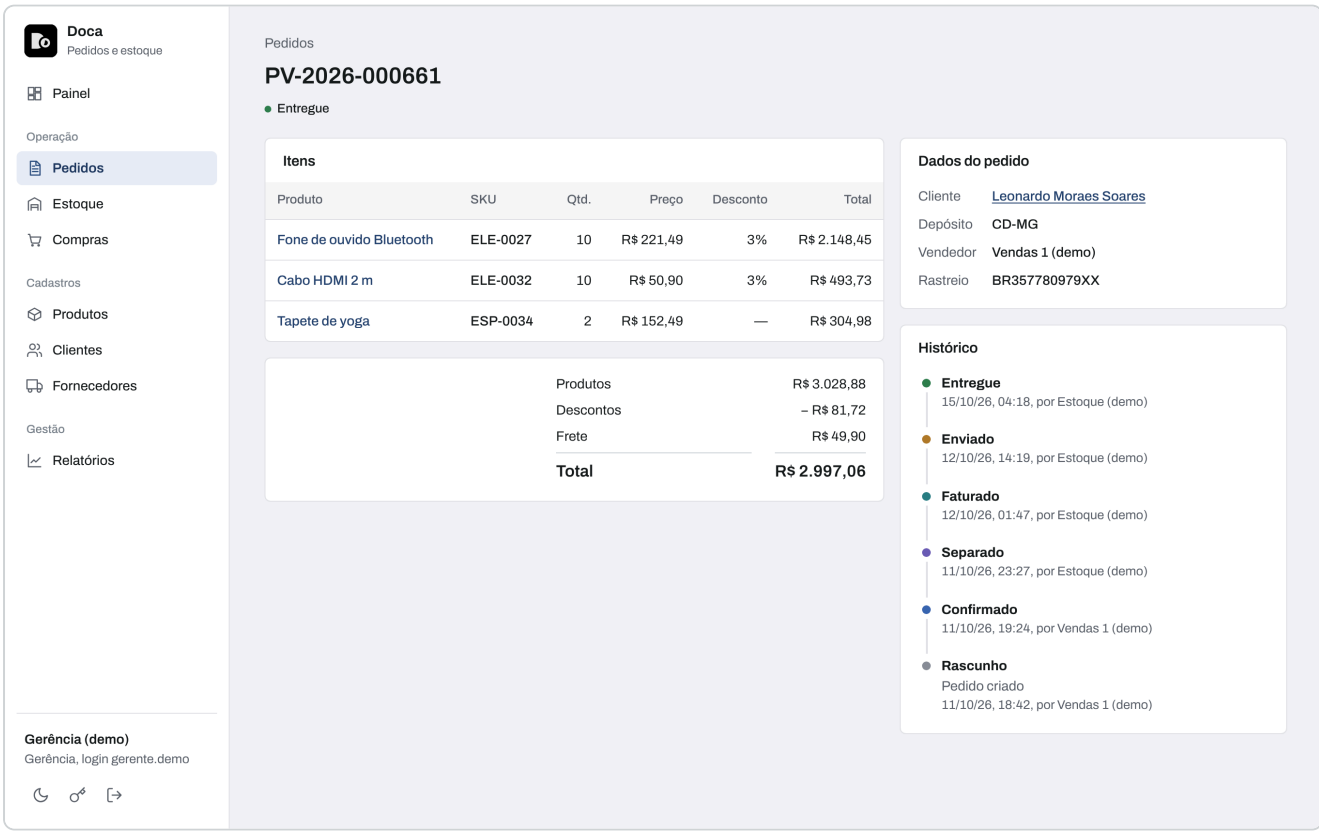
Novo pedido com três itens

Detalhe do pedido

Mostra os itens com preço e desconto, os totais, os dados do pedido e o histórico de cada mudança de situação, com data e responsável. Os botões no topo mudam conforme a situação e o perfil: só aparecem as ações que você pode executar naquele momento.



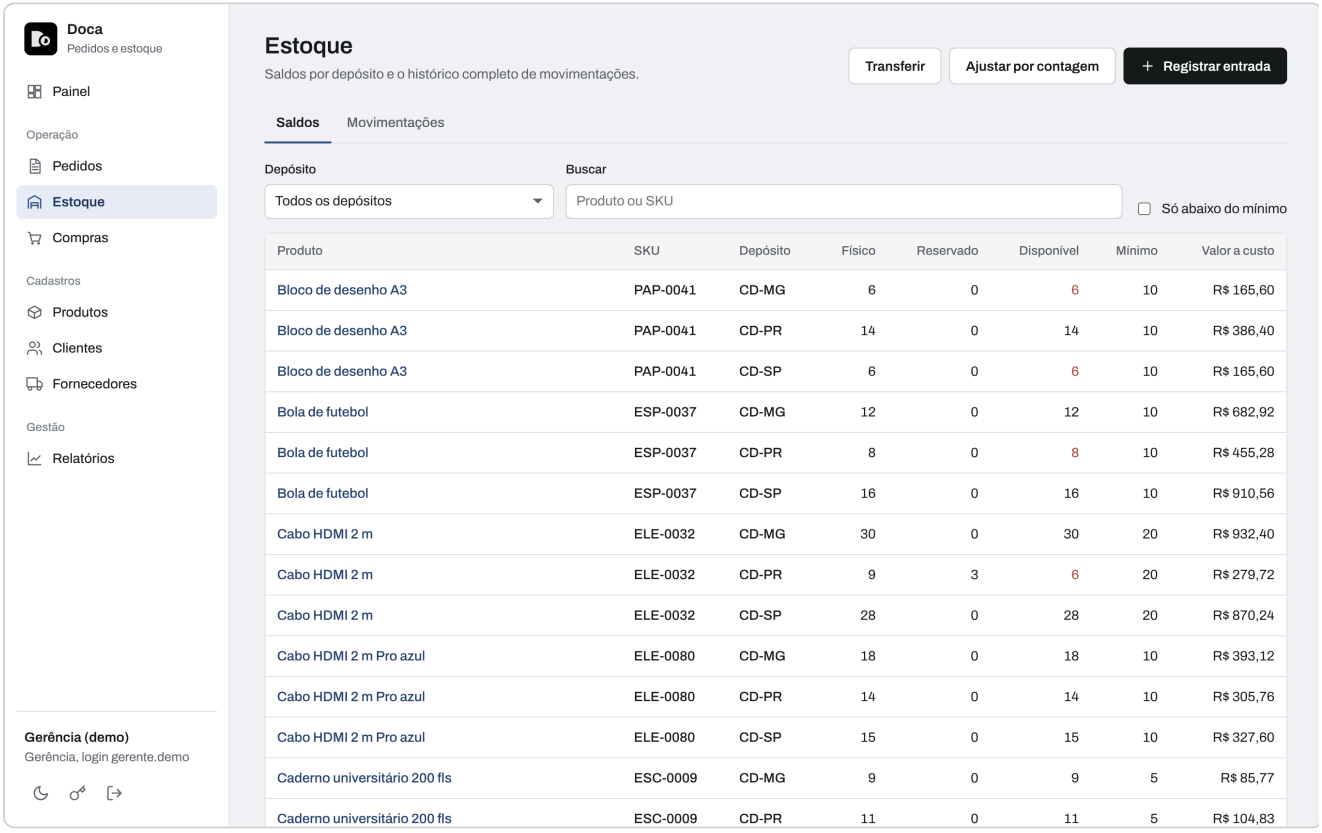
Pedido separado, pronto para faturar



Pedido entregue com o histórico completo

Estoque

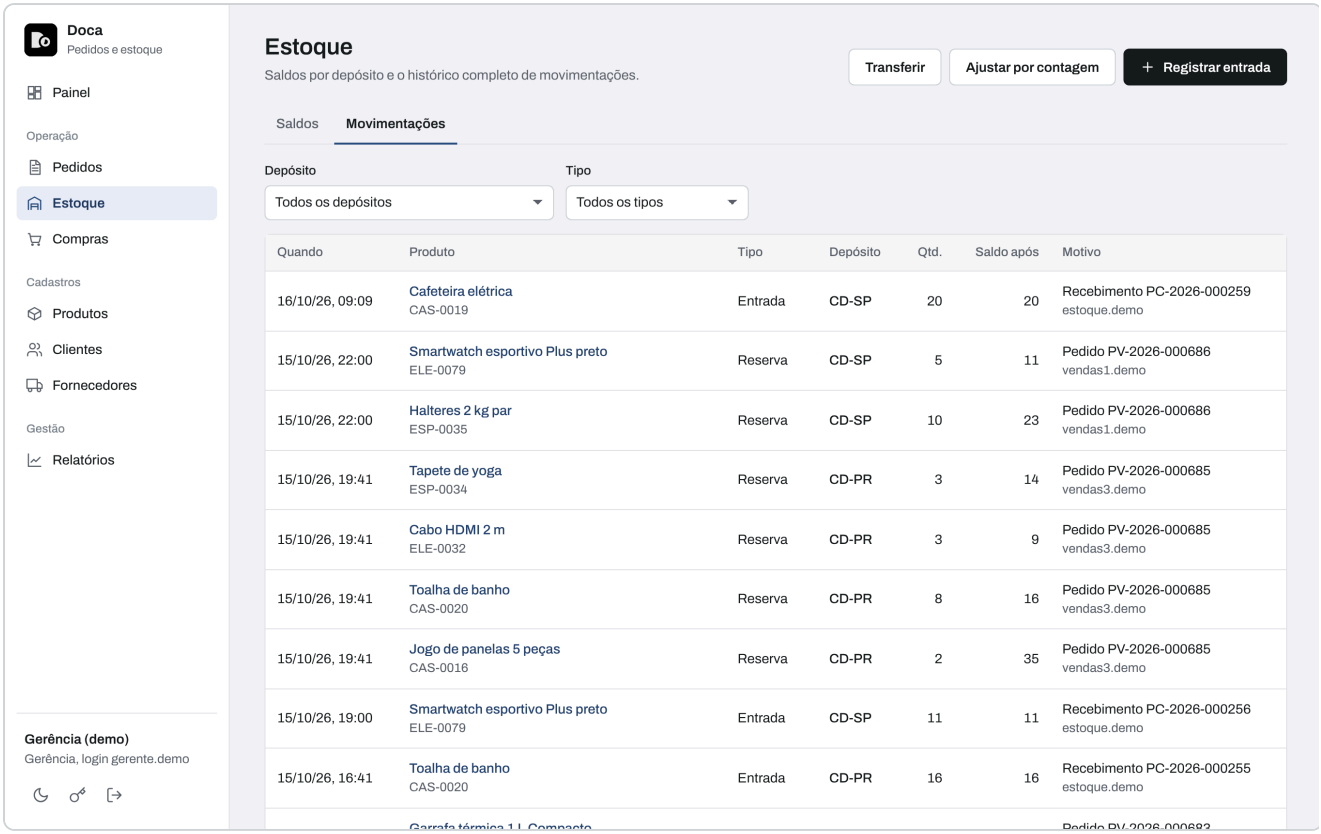
A aba **Saldos** mostra, por produto e depósito, o físico, o reservado, o disponível, o mínimo e o valor a preço de custo, com filtro de itens abaixo do mínimo. Os botões do topo registram entrada, ajuste por contagem e transferência entre depósitos.



Produto	SKU	Depósito	Físico	Reservado	Disponível	Mínimo	Valor a custo
Bloco de desenho A3	PAP-0041	CD-MG	6	0	6	10	R$ 165,60
Bloco de desenho A3	PAP-0041	CD-PR	14	0	14	10	R$ 386,40
Bloco de desenho A3	PAP-0041	CD-SP	6	0	6	10	R$ 165,60
Bola de futebol	ESP-0037	CD-MG	12	0	12	10	R$ 682,92
Bola de futebol	ESP-0037	CD-PR	8	0	8	10	R$ 455,28
Bola de futebol	ESP-0037	CD-SP	16	0	16	10	R$ 910,56
Cabo HDMI 2 m	ELE-0032	CD-MG	30	0	30	20	R$ 932,40
Cabo HDMI 2 m	ELE-0032	CD-PR	9	3	6	20	R$ 279,72
Cabo HDMI 2 m	ELE-0032	CD-SP	28	0	28	20	R$ 870,24
Cabo HDMI 2 m Pro azul	ELE-0080	CD-MG	18	0	18	10	R$ 393,12
Cabo HDMI 2 m Pro azul	ELE-0080	CD-PR	14	0	14	10	R$ 305,76
Cabo HDMI 2 m Pro azul	ELE-0080	CD-SP	15	0	15	10	R$ 327,60
Caderno universitário 200 fls	ESC-0009	CD-MG	9	0	9	5	R$ 85,77
Caderno universitário 200 fls	ESC-0009	CD-PR	11	0	11	5	R$ 104,83

Saldos por depósito

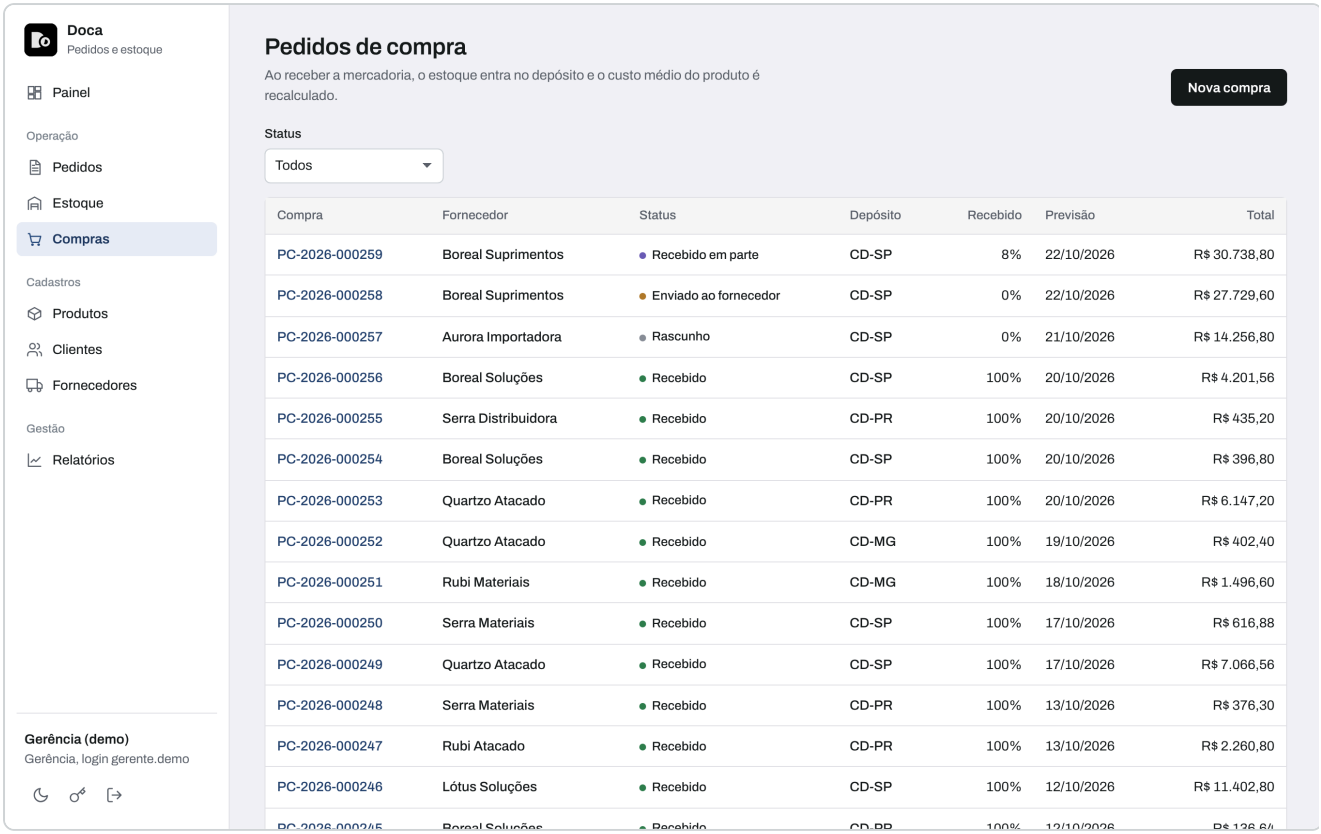
A aba **Movimentações** é o livro do estoque: cada linha diz o tipo, a quantidade, o saldo depois da operação, quem fez e a origem (pedido, compra, transferência, inventário ou manual).



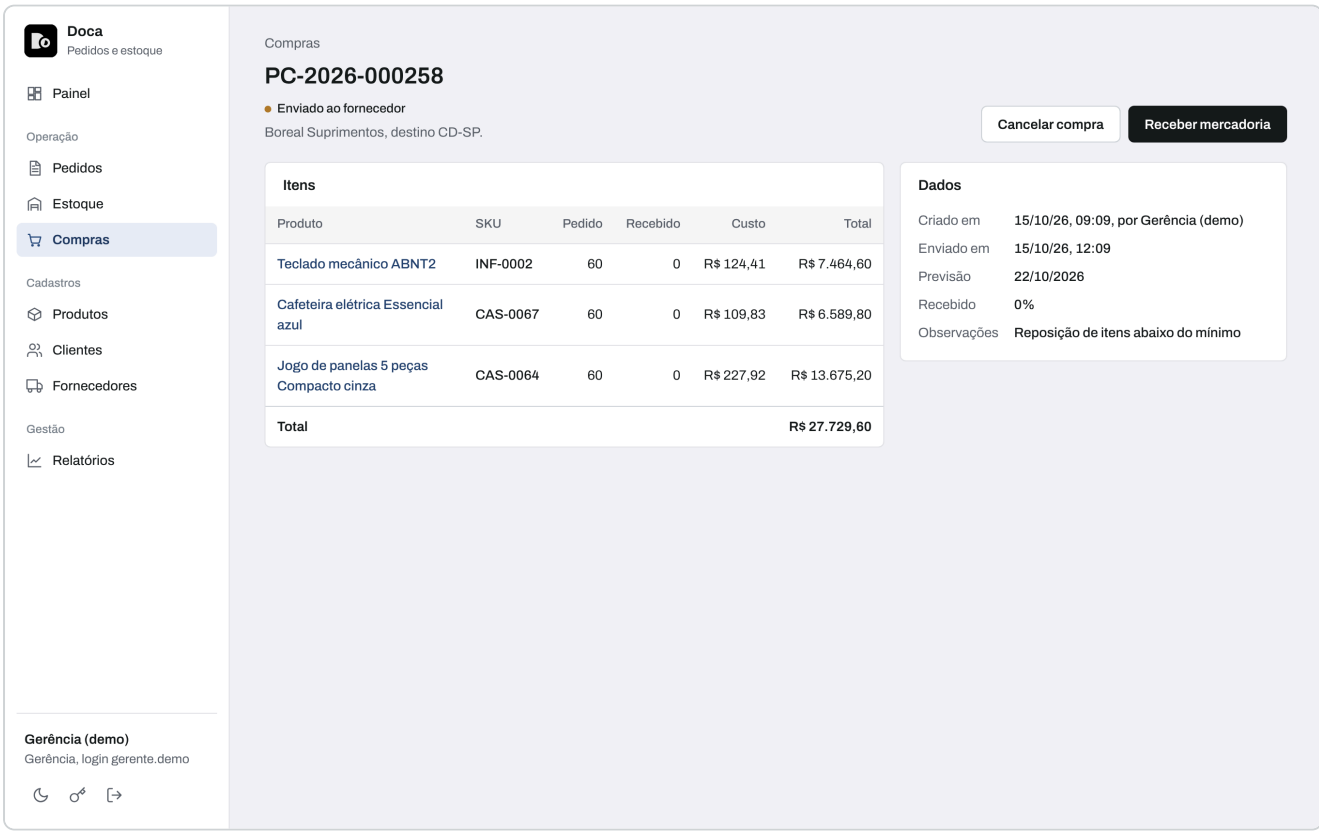
Livro de movimentações

Compras

Os pedidos de compra seguem a ordem rascunho, enviado ao fornecedor, recebido parcial e recebido. A lista mostra o percentual já recebido de cada compra. No detalhe, **Receber mercadoria** aceita quantidades parciais e recalcula o custo médio dos produtos.



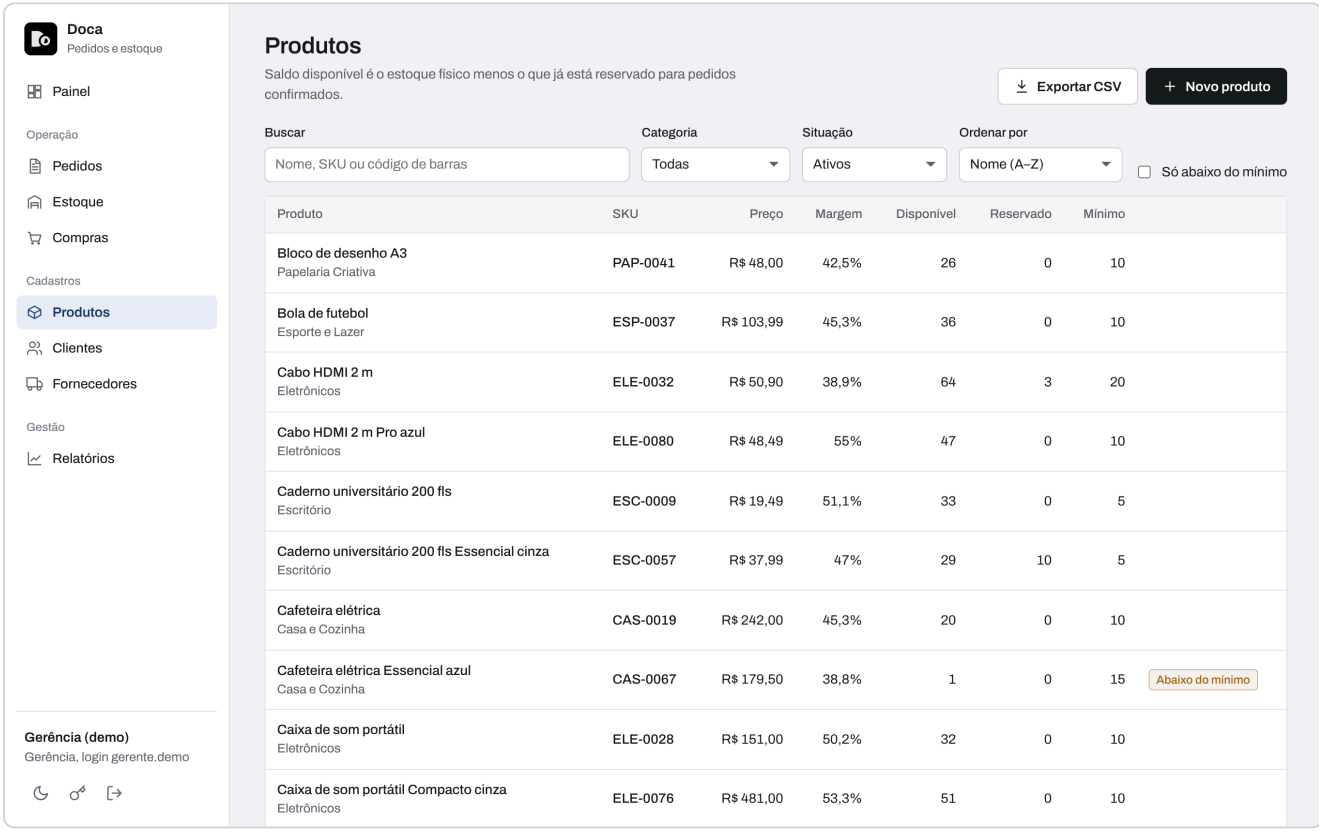
Pedidos de compra



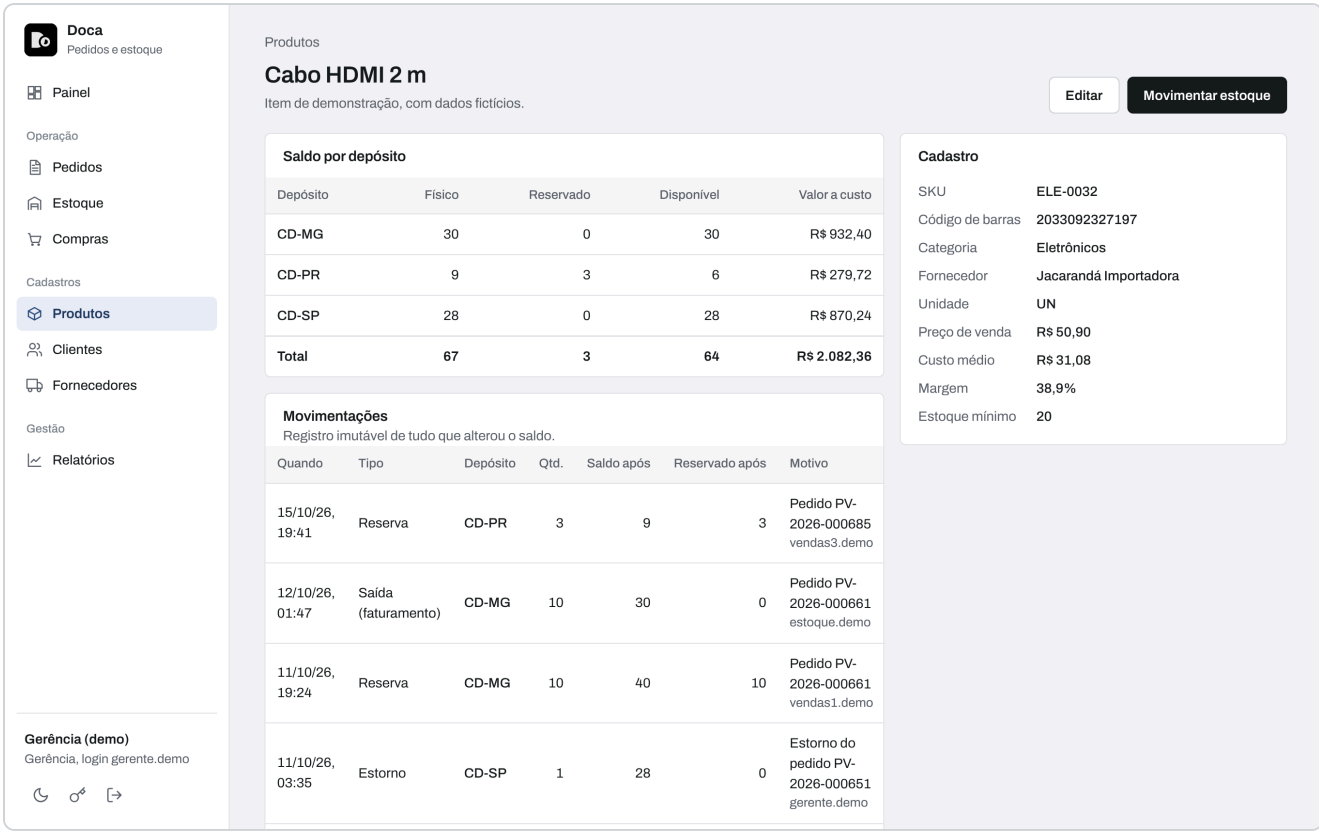
Compra enviada ao fornecedor, aguardando recebimento

Produtos

Busca por nome, SKU ou código de barras, com filtro por categoria, situação e itens abaixo do mínimo, e exportação em CSV. O detalhe do produto reúne o saldo por depósito, as últimas movimentações e o cadastro, com botões para editar e movimentar o estoque.



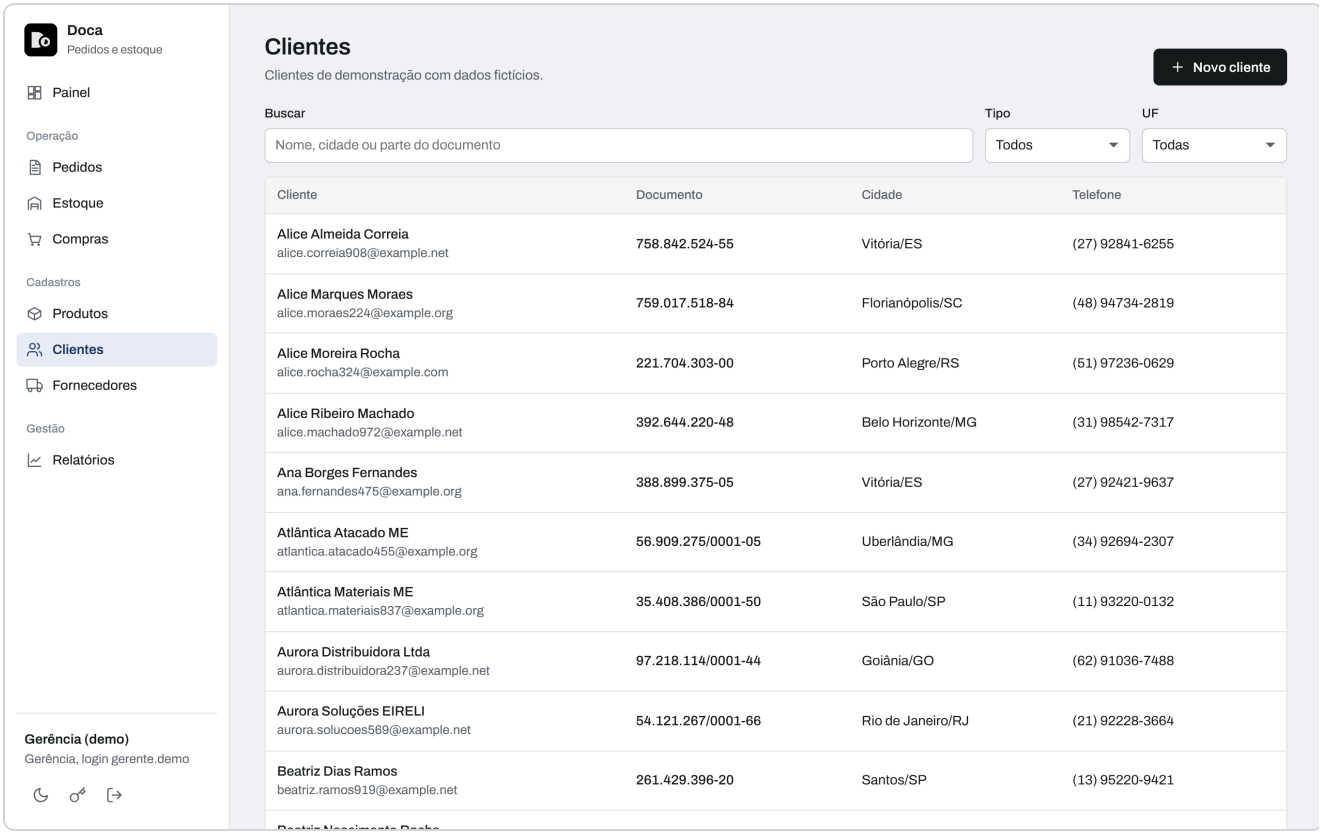
Lista de produtos



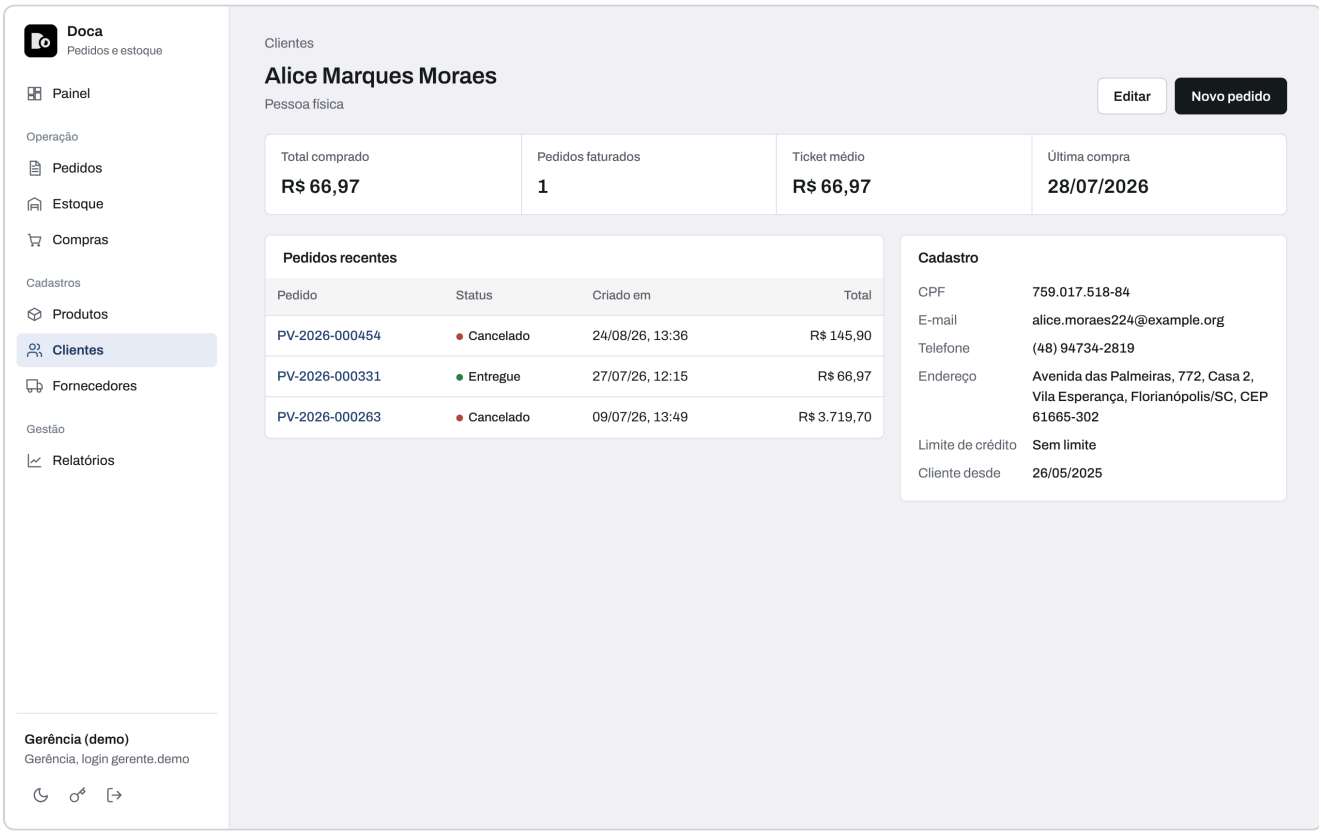
Detalhe do produto

Clientes e fornecedores

Clientes PF e PJ com documento validado, cidade, UF e limite de crédito. Para os perfis operacionais o CPF aparece mascarado; revelar o número completo fica registrado na auditoria. O detalhe do cliente mostra total comprado, pedidos faturados, ticket médio, última compra e os pedidos recentes.

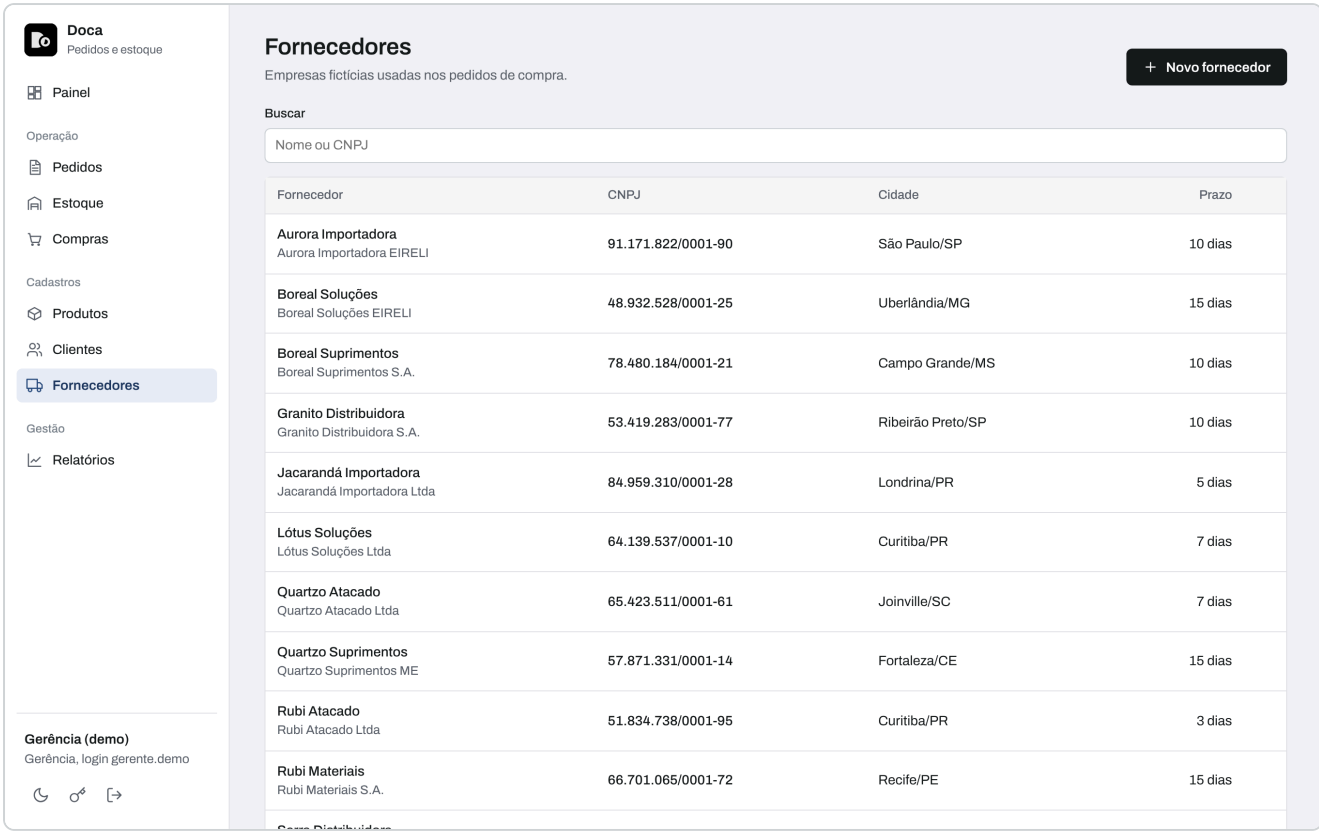


Lista de clientes



Detalhe do cliente

Fornecedores têm CNPJ, contato, cidade e prazo de entrega em dias, usado na sugestão de compra.

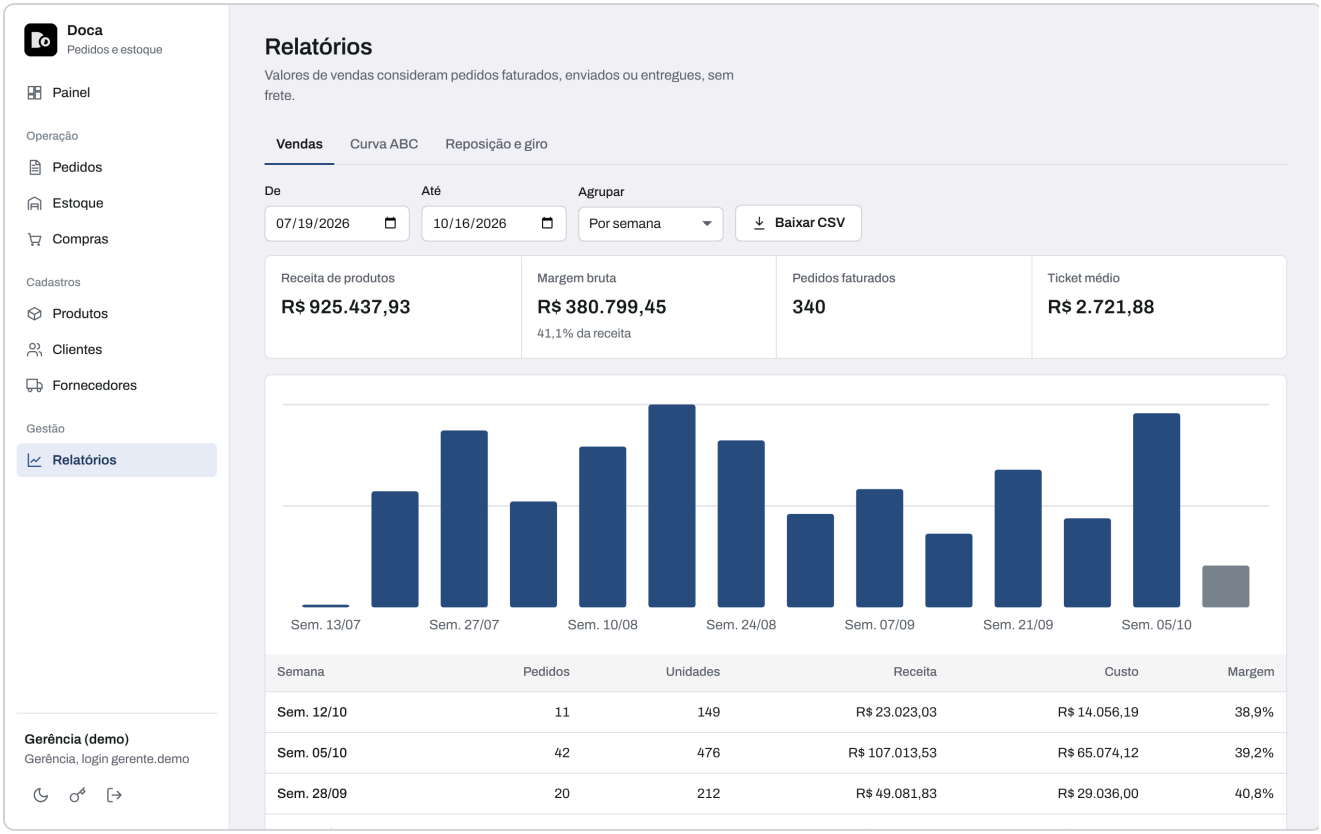


Fornecedores

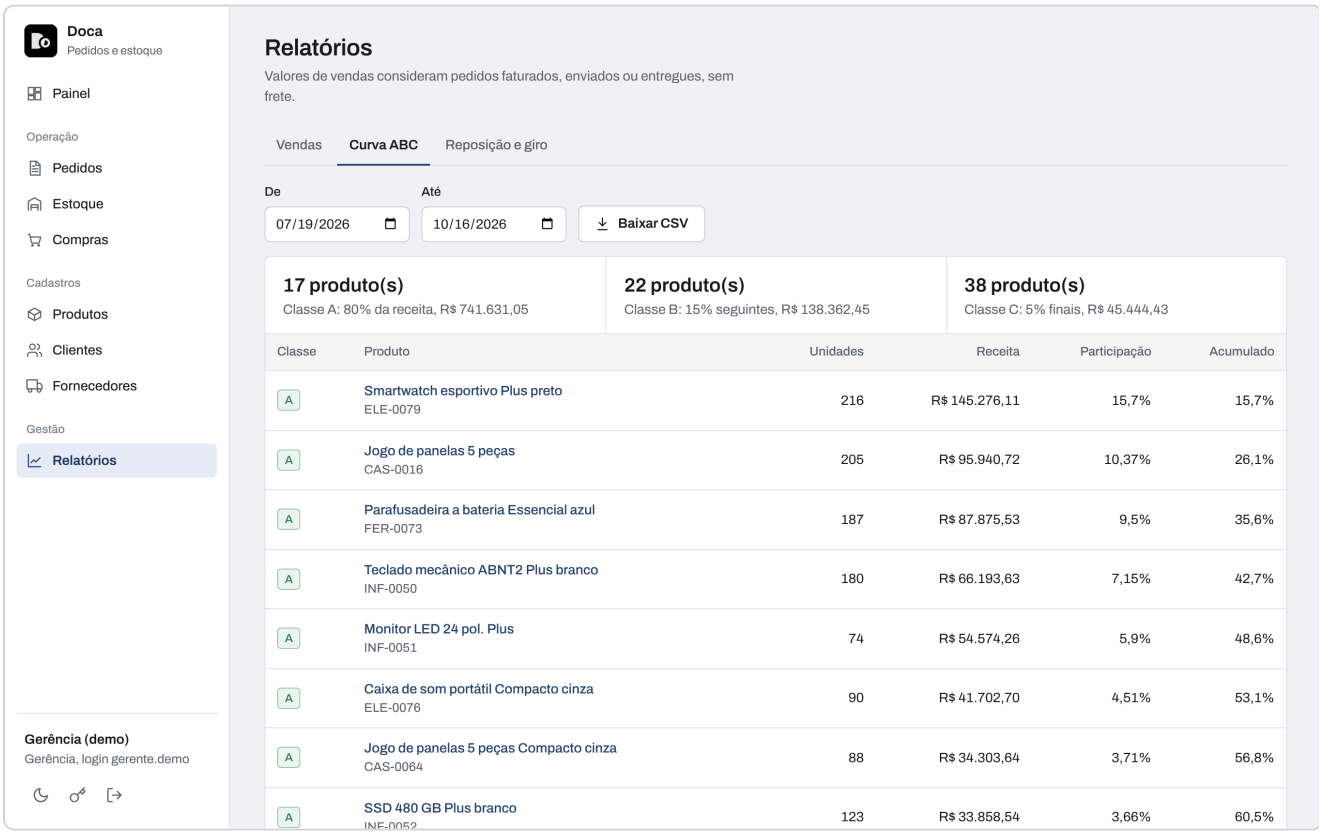
Relatórios

Os valores de vendas consideram pedidos faturados, enviados ou entregues, sem frete. Todas as abas aceitam período e exportação em CSV.

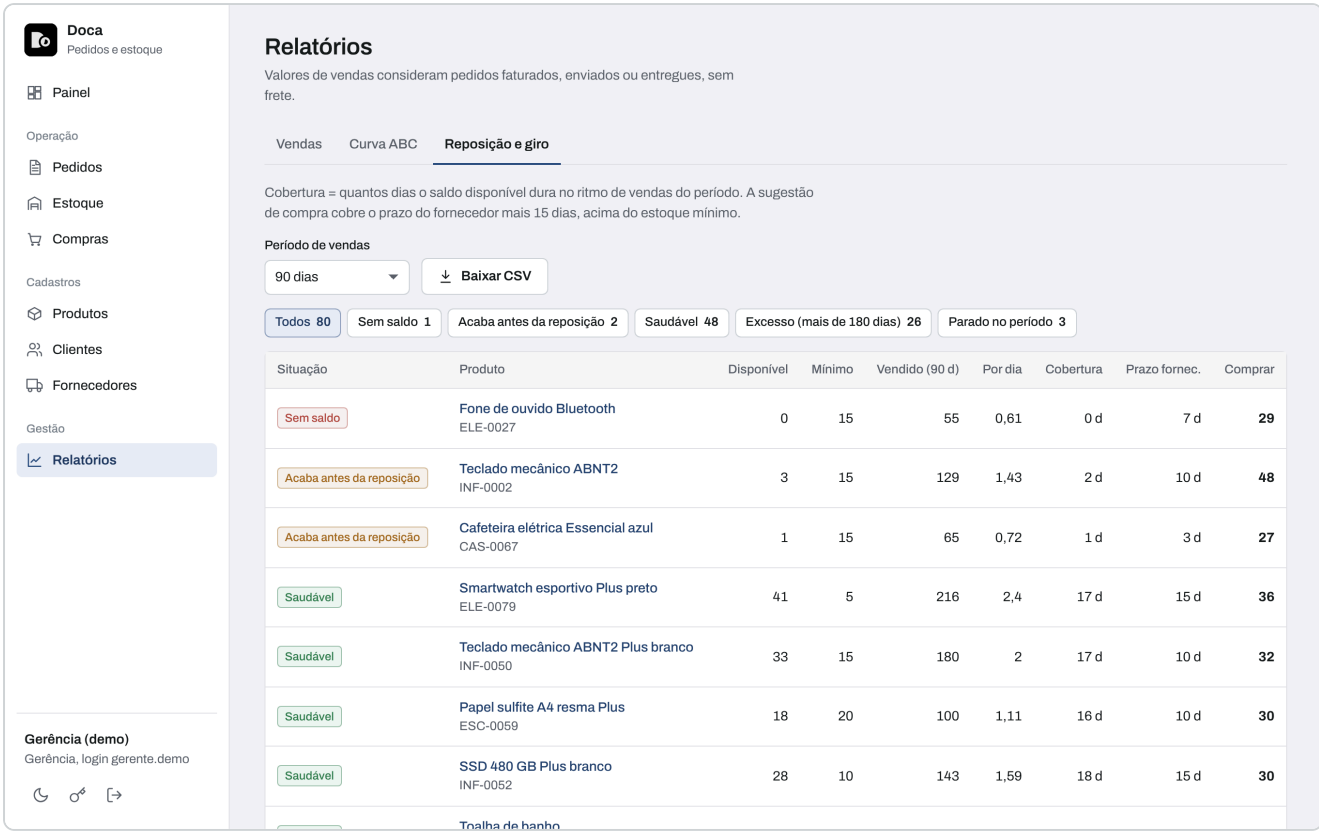
- **Vendas:** receita, margem bruta, pedidos e ticket médio, agrupados por dia, semana, mês, categoria, produto, vendedor, cliente ou UF.
- **Curva ABC:** classifica os produtos por participação na receita (A, B e C) com o acumulado.
- **Reposição e giro:** disponível, vendas no período, giro, cobertura em dias e quantidade sugerida para compra, considerando o prazo do fornecedor.



Vendas por semana



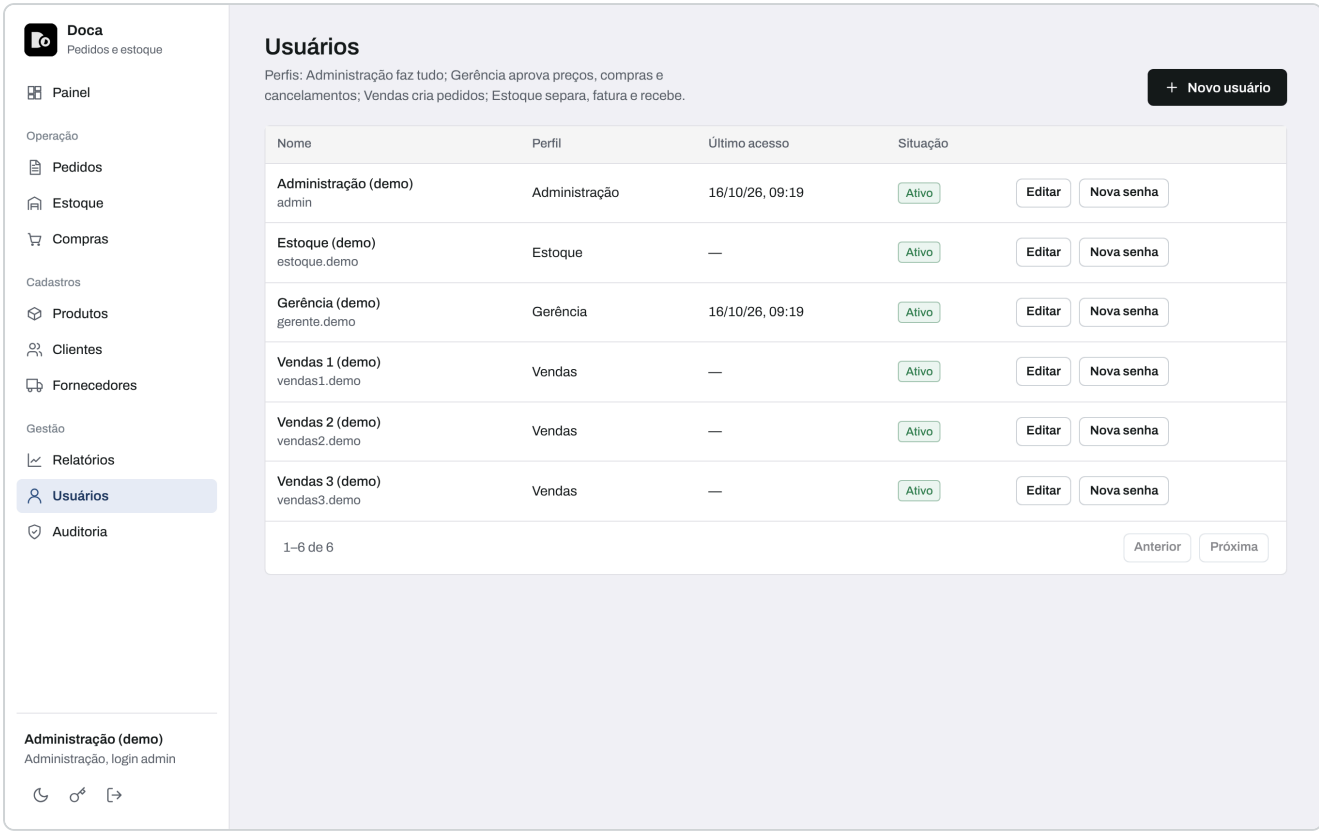
Curva ABC



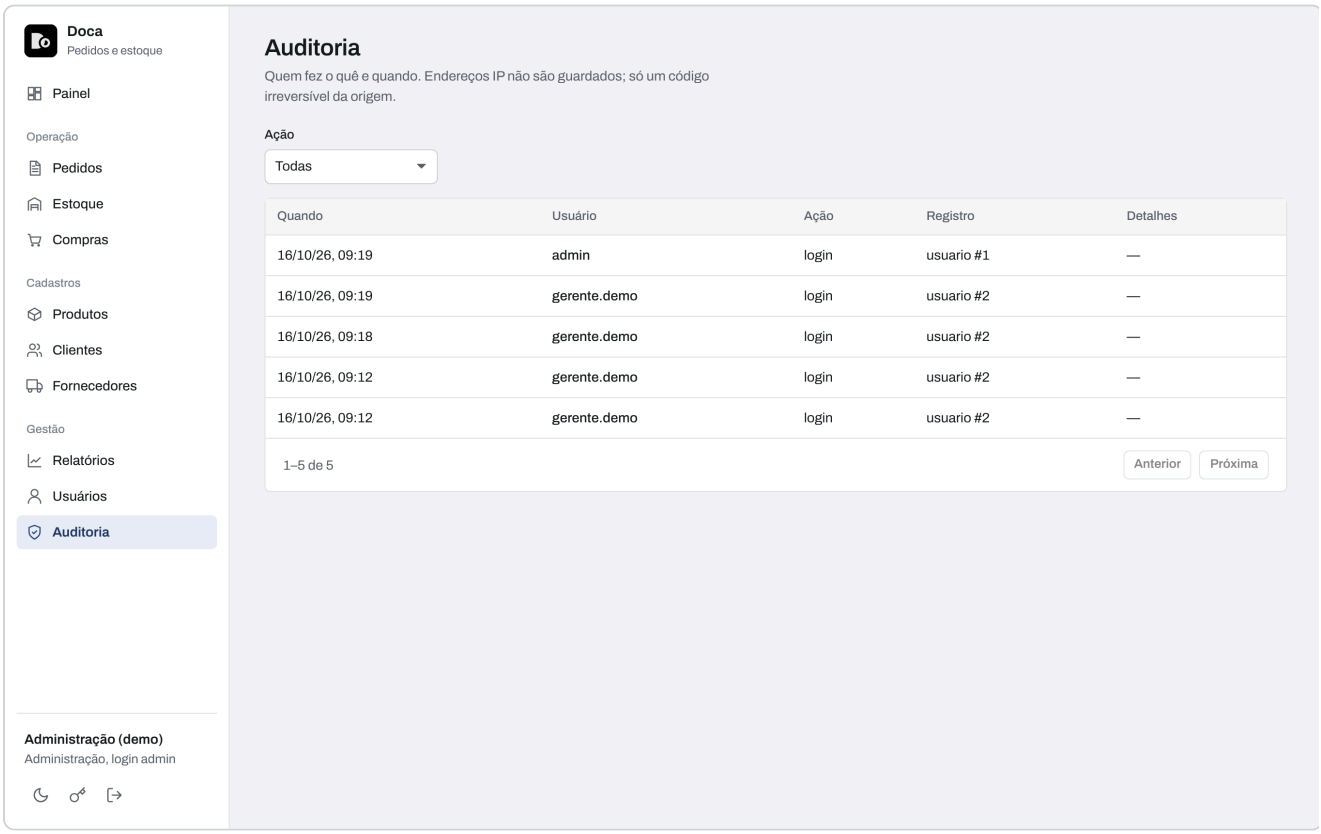
Reposição e giro

Usuários e auditoria

Só a administração acessa. Em **Usuários** é possível criar contas (com senha provisória), mudar o perfil, desativar e redefinir senha. Em **Auditoria** fica a trilha de quem fez o quê: entradas no sistema, mudanças de cadastro, revelação de documento e ações administrativas, sempre sem endereço IP.



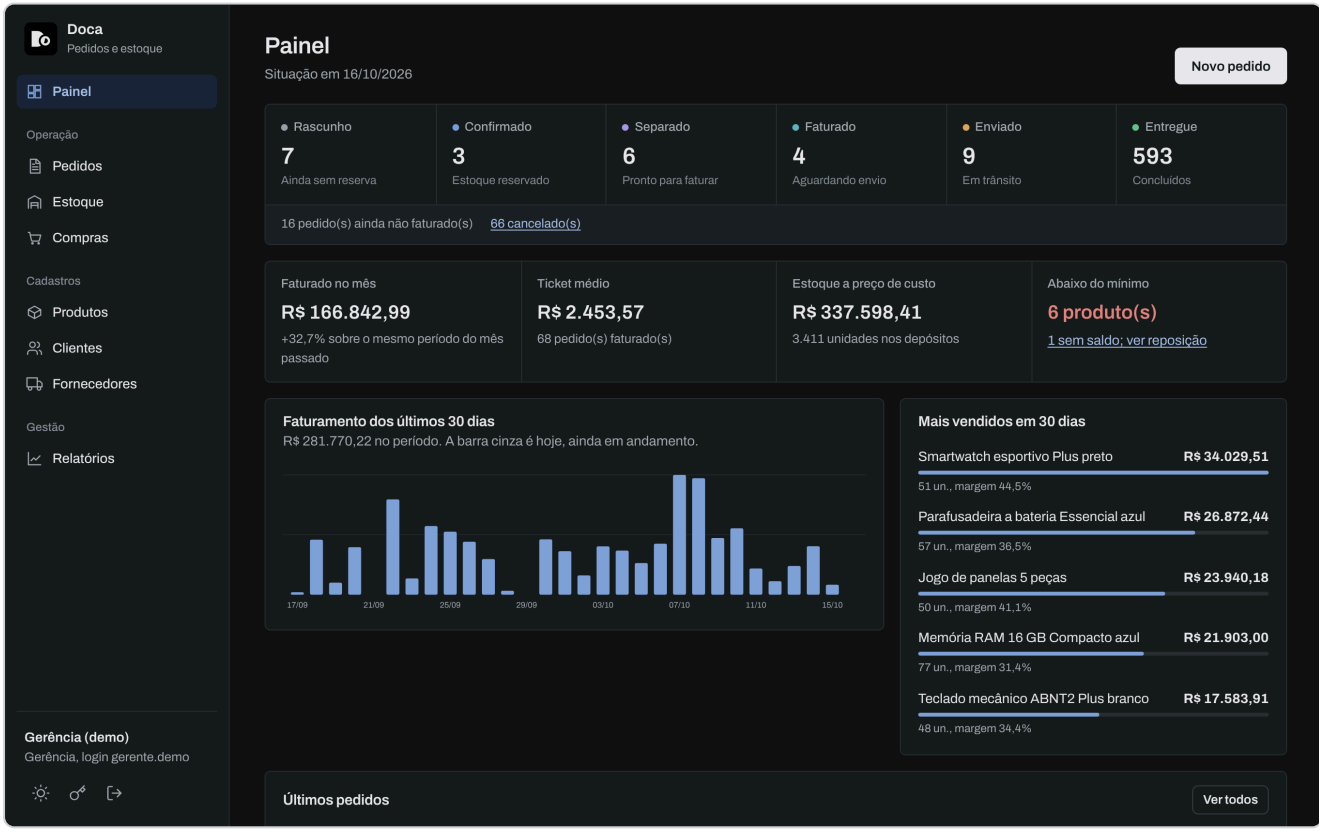
Usuários



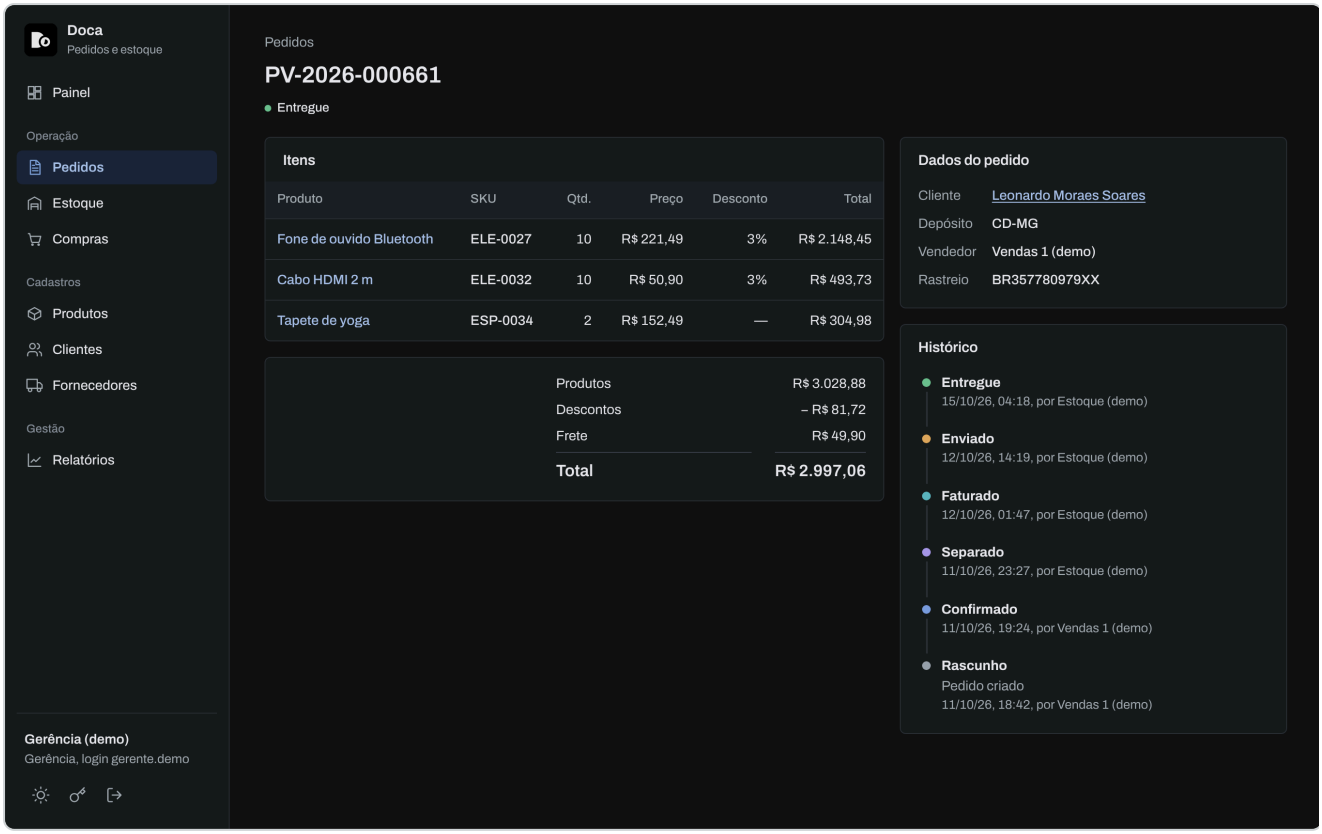
Trilha de auditoria

Tema escuro

O ícone de lua no rodapé do menu alterna entre claro e escuro. A escolha fica guardada no navegador; na primeira visita o painel segue o tema do sistema operacional. O tema escuro mantém a mesma estrutura, com fundo quase preto, painéis grafite e o destaque em azul mais claro.



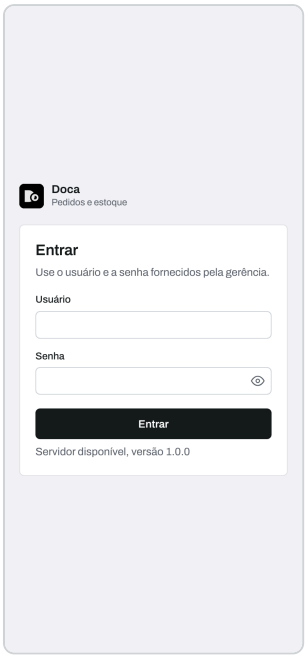
Painel no tema escuro



Detalhe de pedido no tema escuro

No celular

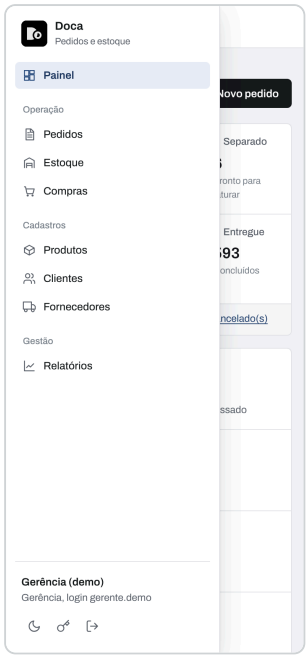
Em telas pequenas o menu lateral vira uma gaveta aberta pelo botão no topo, os pedidos por etapa e os indicadores se reorganizam em colunas e as tabelas rolam na horizontal.



Entrada



Painel



Menu aberto



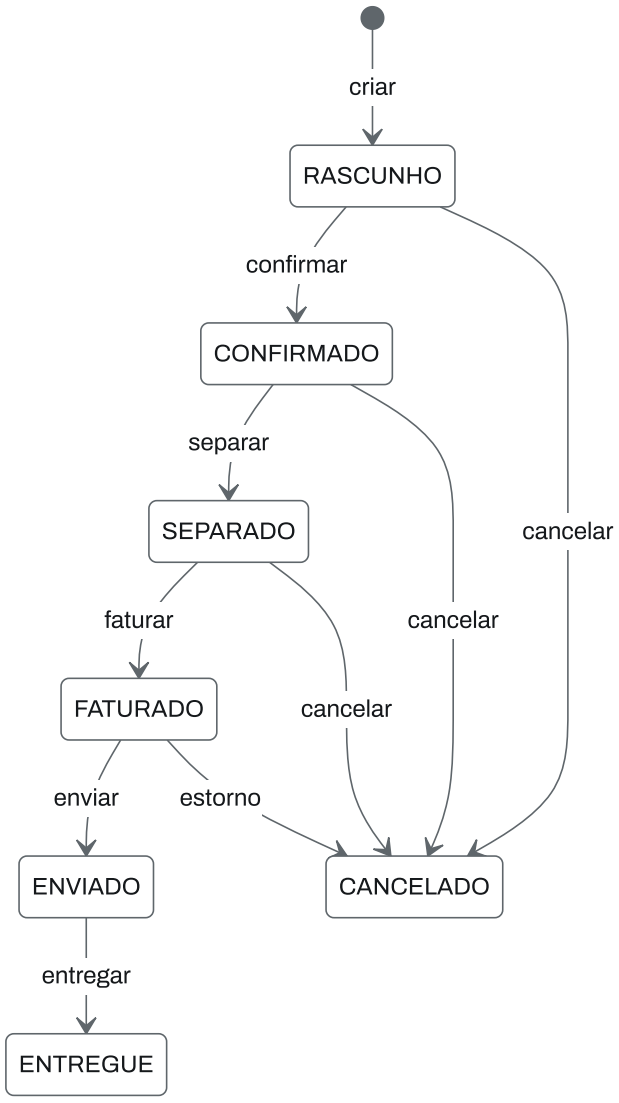
Pedido

Capítulo 6

Pedidos, estoque e compras

As regras de negócio ficam em `app/servicos/`, sem nada de HTTP. As rotas abrem a transação, chamam o serviço e fazem `commit` uma única vez: se qualquer regra falhar, nada é gravado.

Ciclo do pedido

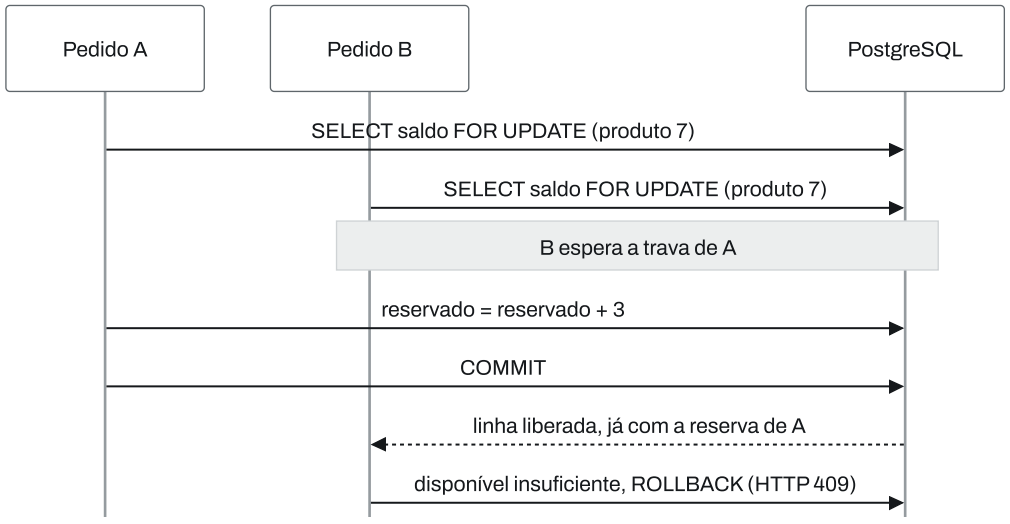


Situações do pedido de venda

Situação	O que acontece com o estoque	Quem avança
Rascunho	Nada. Itens, cliente e frete podem ser editados.	Vendas, gerência, administração
Confirmado	Todos os itens são reservados no depósito de saída.	Estoque, gerência, administração

Situação	O que acontece com o estoque	Quem avança
Separado	Continua reservado.	Estoque, gerência, administração
Faturado	Sai do físico e do reservado (baixa definitiva).	Estoque, gerência, administração
Enviado	Registra o código de rastreio.	Estoque, gerência, administração
Entregue	Fim do ciclo.	—
Cancelado	Antes do faturamento libera a reserva; depois, só a gerência cancela e o estoque volta (estorno).	—

Reserva sem venda dupla



Dois pedidos disputando a última unidade

- **Tudo ou nada:** ao confirmar, todos os itens são reservados ou nenhum.
- **Ordem fixa das travas:** as linhas de saldo são travadas sempre em ordem crescente de produto, então dois pedidos com os mesmos itens nunca travam um ao outro em ciclo (*deadlock*).
- **Disponível = físico − reservado.** O banco garante por `CHECK` que nenhum saldo fica negativo e que a reserva nunca passa do físico. É a última linha de defesa caso algum código futuro esqueça a regra.
- **Prova automatizada:** `tests/test_concorrencia.py` dispara 20 confirmações simultâneas para 5 unidades e exige exatamente 5 sucessos.

Regras de valor

Regra	Como funciona
Desconto por perfil	Vendas até 10%, gerência até 30%, administração sem limite. Acima disso o pedido é recusado com o SKU e o limite.

Regra	Como funciona
Limite de crédito	Se o pedido somado aos pedidos em aberto passar do limite do cliente, vendas é barrado e a gerência pode liberar; a liberação fica no histórico.
Custo congelado	O custo médio do produto é gravado no item no momento da venda, então a margem histórica não muda depois.
Custo médio ponderado	Recalculado a cada recebimento de compra.
Idempotência	`POST /pedidos` aceita o cabeçalho `Idempotency-Key`; repetir a requisição devolve o mesmo pedido.
Versão otimista	Produtos e pedidos têm número de versão: duas pessoas editando o mesmo registro não sobrescrevem uma à outra (HTTP 409).

Tipos de movimentação

Tipo	Quando acontece
`ENTRADA`	Entrada manual ou recebimento de compra
`SAIDA`	Faturamento do pedido
`RESERVA` / `LIBERACAO_RESERVA`	Confirmação e cancelamento antes do faturamento
`AJUSTE_POSITIVO` / `AJUSTE_NEGATIVO`	Ajuste por contagem (inventário)
`TRANSFERENCIA_SAIDA` / `TRANSFERENCIA_ENTRADA`	Transferência entre depósitos
`ESTORNO`	Cancelamento de pedido já faturado

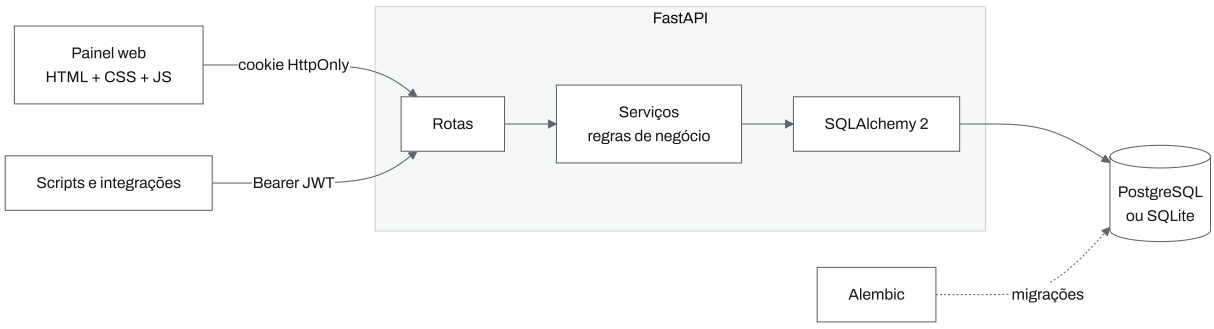
Ciclo da compra

Rascunho → Enviado ao fornecedor → Recebido parcial → Recebido. A compra pode ser cancelada enquanto estiver em rascunho ou enviada; depois do primeiro recebimento, não. Cada recebimento gera uma `ENTRADA` no depósito de destino e atualiza o custo médio.

Capítulo 7

Arquitetura

Uma aplicação única: o FastAPI serve a API em `/api/v1` e os arquivos do painel na mesma porta.



Visão geral

Camadas do código

Pasta	Responsabilidade
`app/rotas/`	HTTP: autenticação, permissão por perfil, paginação e formato da resposta
`app/servicos/`	Regras de negócio, sem nada de HTTP (reaproveitadas pelo gerador de dados fictícios)
`app/modelos/`	Tabelas e restrições do banco (`CHECK` , chaves únicas, índices)
`app/esquemas/`	Validação de entrada e formato de saída com Pydantic
`web/js/views/`	Uma tela por arquivo: painel, pedidos, estoque, compras, produtos, clientes, fornecedores, relatórios e administração

Decisões principais

- **Dinheiro como inteiro de centavos.** O tipo `Dinheiro` (`app/database.py`) grava `BIGINT` e devolve `Decimal` com duas casas. Funciona igual em PostgreSQL e SQLite, e somas no banco continuam exatas.
- **Saldo e livro.** `saldo_estoque` tem o número atual, rápido de ler; `movimentacao_estoque` é o histórico imutável, com o saldo depois de cada operação. Toda alteração passa por `app/servicos/estoque.py` , que grava as duas coisas na mesma transação.
- **Máquina de estados declarativa.** `TRANSICOES` em `app/servicos/pedidos.py` define, para cada ação, de quais situações ela parte, para qual vai e quais perfis podem executá-la. A mesma tabela alimenta o campo `acoes_disponiveis` , usado pelo painel para mostrar só os botões permitidos.
- **Travas só no saldo.** `FOR UPDATE OF saldo_estoque` trava só a tabela de saldo, mesmo com relacionamentos carregados por `LEFT JOIN` .
- **Tokens revogáveis sem lista negra.** O JWT leva a versão do token do usuário; aumentar `versao_token` invalida todas as sessões dele.

- **Fuso horário.** Datas são gravadas em UTC (`DataHoraUTC`). Relatórios agrupam por dia no fuso `America/Sao_Paulo` .
- **Interface sem build.** HTML, CSS e JavaScript em módulos, servidos como arquivos estáticos. Nada é carregado de fora.

Erros

Toda resposta de erro tem o mesmo formato, e o painel mostra a mensagem e os detalhes junto do campo ou da ação:

```
{ "erro": { "codigo": "estoque_insuficiente",
            "mensagem": "Estoque disponível insuficiente para confirmar o pedido.",
            "detalhes": [{ "sku": "INF-0003", "disponivel": 4, "solicitado": 6 }] } }
```

Capítulo 8

API

Documentação interativa em **http://127.0.0.1:8000/docs** (Swagger) e `/redoc`. Todas as rotas usam o prefixo `/api/v1` e, exceto entrada e saúde, exigem login.

Método	Rota	Função
POST	`/auth/login`	Entrar (cookie para o painel e token Bearer para a API)
POST	`/auth/logout` · `/auth/sair-de-todos`	Sair desta sessão ou de todas
POST	`/auth/trocar-senha`	Trocar a própria senha
GET	`/auth/eu`	Usuário da sessão atual
GET / POST	`/pedidos`	Listar e criar pedidos (aceita `Idempotency-Key`)
GET	`/pedidos/exportar`	Pedidos filtrados em CSV
GET / PATCH	`/pedidos/{id}`	Ver e editar um pedido
PUT	`/pedidos/{id}/itens`	Substituir os itens de um rascunho
POST	`/pedidos/{id}/confirmar` · `separar` · `faturar` · `enviar` · `entregar` · `cancelar`	Avançar ou cancelar o pedido
GET	`/estoque/saldos` · `/estoque/movimentacoes`	Saldos e livro de movimentações
POST	`/estoque/entradas` · `ajustes` · `transferencias`	Movimentar estoque
GET / POST / PUT	`/depositos`	Depósitos
GET / POST	`/compras`	Listar e criar compras
POST	`/compras/{id}/enviar` · `receber` · `cancelar`	Avançar a compra
GET / POST / PATCH	`/produtos`	Produtos, com `/exportar`, `/{id}/saldos` e `/{id}/movimentacoes`
GET / POST / PUT	`/categorias` · `/fornecedores`	Catálogo
GET / POST / PUT	`/clientes`	Clientes, com `/{id}/resumo` e `/{id}/documento` (auditado)
GET / POST / PATCH	`/usuarios`	Usuários, com `/{id}/redefinir-senha` (administração)
GET	`/relatorios/dashboard` · `vendas` · `curva-abc` · `giro-estoque`	Relatórios (aceitam `formato=csv`)
GET	`/auditoria`	Trilha de auditoria (administração)
GET	`/saude`	Verificação de saúde, sem dados sensíveis

Exemplo de uso

```
# 1. Entrar e guardar o token
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"usuario": "<seu-usuario>", "senha": "<sua-senha>"}' \
  | python -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

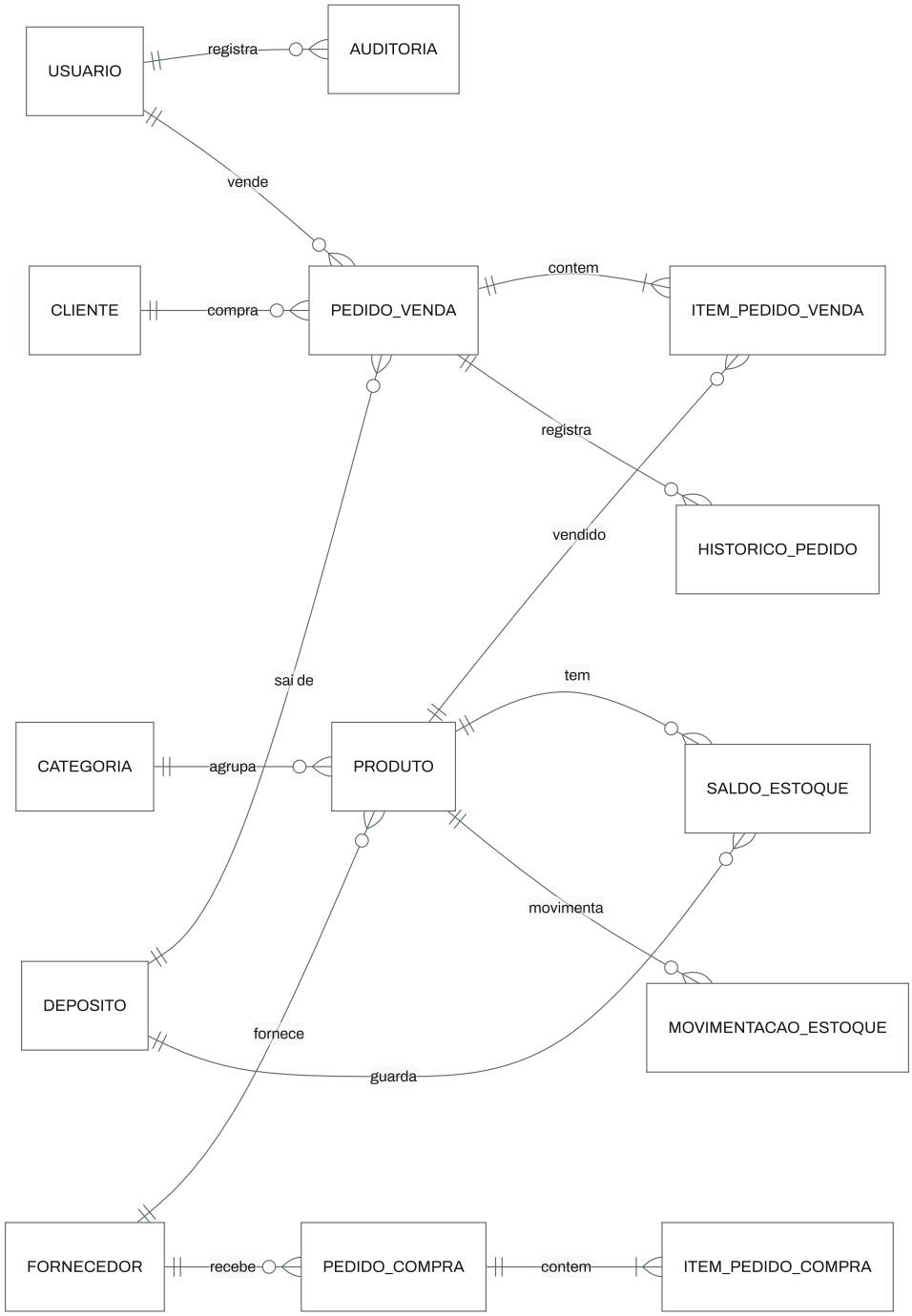
# 2. Criar um pedido (repetir com a mesma chave não duplica)
curl -X POST http://127.0.0.1:8000/api/v1/pedidos \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -H "Idempotency-Key: pedido-exemplo-001" \
  -d '{"cliente_id": 1, "deposito_id": 1, "frete": "19.90",
       "itens": [{"produto_id": 3, "quantidade": 2, "desconto_percentual": 5}]}'

# 3. Confirmar (reserva o estoque)
curl -X POST http://127.0.0.1:8000/api/v1/pedidos/1/confirmar \
  -H "Authorization: Bearer $TOKEN"
```

Capítulo 9

Banco de dados

O Doca cria as tabelas sozinho na primeira execução (SQLite) ou pelas migrações do Alembic (Docker e produção). As regras críticas também estão no próprio banco, em restrições `CHECK` e chaves únicas.



Relacionamentos principais

Tabelas

Tabela	O que guarda	Restrições importantes
`produto`, `categoria`, `fornecedor`, `deposito`	Cadastros	SKU e EAN únicos; versão otimista no produto
`cliente`	Pessoas físicas e jurídicas	Documento único e validado; limite de crédito
`saldo_estoque`	Físico e reservado por produto e depósito	`quantidade >= 0`; `0 <= reservado <= quantidade`
`movimentacao_estoque`	Livro imutável de tudo que alterou um saldo	`quantidade > 0`; guarda saldo e reservado depois da operação
`pedido_venda`	Cabeçalho do pedido	Número `PV-AAAA-000001`; chave de idempotência única por vendedor; versão
`item_pedido_venda`	Itens com preço, desconto e custo congelados	Quantidade positiva
`historico_pedido`	Cada mudança de situação	Data, responsável e observação
`pedido_compra`, `item_pedido_compra`	Compras e quanto já foi recebido	Recebido nunca passa do pedido
`usuario`, `auditoria`	Acesso por perfil e trilha de ações	Login único; origem guardada só como HMAC

Migrações

A estrutura inicial fica em `alembic/versions/`. Para mudar o banco:

```
alembic revision --autogenerate -m "descreva a mudança"
alembic upgrade head
alembic check   # falha se os modelos e as migrações divergirem
```

Usar PostgreSQL fora do Docker

1. Crie um banco e um usuário no seu PostgreSQL.
2. No `.env`, defina `DATABASE_URL=postgresql://<usuario>:<senha>@<servidor>:5432/<banco>` e `CRIAR_TABELAS=false`.
3. Rode `alembic upgrade head` para criar as tabelas.
4. Opcional: `python -m scripts.popular_banco` para os dados de demonstração.

Capítulo 10

Segurança e privacidade

O Doca foi pensado para ser publicado no GitHub sem expor ninguém: nem quem desenvolve, nem quem usa.

Área	O que o sistema faz
Senhas	scrypt com sal individual, comparação em tempo constante e mesma resposta para usuário inexistente ou senha errada.
Tentativas	Bloqueio temporário após 5 tentativas erradas (por origem e por usuário) e limite geral de requisições por minuto.
Sessão	Cookie HttpOnly + SameSite=Strict; escrita vinda de outro site é recusada pelo middleware (proteção CSRF).
Revogação	Trocar senha, desativar usuário ou mudar o perfil encerra todas as sessões.
Cabeçalhos	CSP restritiva (sem scripts ou estilos de terceiros), `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: no-referrer`, `Permissions-Policy` e HSTS em HTTPS.
Endereço IP	Nunca é gravado. A auditoria guarda só um código HMAC irreversível da origem e os logs não registram IP.
LGPD	CPF mascarado para perfis operacionais; revelar o documento fica registrado (princípio da necessidade).
Consultas	Sempre parametrizadas pelo ORM, buscas com curingas escapados e validação de entrada com Pydantic.
Terceiros	A interface não carrega nada de fora: sem CDNs, Google Fonts ou analytics.
Segredos	Nenhum no repositório. `.env`, `dados/` e bancos locais estão no `.gitignore`.
Servidor	Escuta só em `127.0.0.1` e não informa a versão do Uvicorn; o Docker publica as portas só para o próprio computador e roda com usuário sem privilégios.

Capítulo 11

Testes, CI e automação

Testes

```
pip install -r requirements-dev.txt
pytest                                  # SQLite em memória
pytest --cov                            # com cobertura
TEST_DATABASE_URL=postgresql://<usuario>:<senha>@localhost:5432/<banco_de_teste> pytest
ruff check . && ruff format --check .
alembic upgrade head && alembic check
```

Arquivo	O que cobre
`test_auth.py`	Hash de senha, força da senha, cookie HttpOnly, bloqueio por tentativas, token adulterado, revogação, usuário desativado e proteção CSRF
`test_pedidos.py`	Totais e numeração, ciclo completo, reserva atômica, cancelamento e estorno, permissões, desconto por perfil, crédito, idempotência e CSV
`test_estoque.py`	Custo médio, entradas, ajustes, transferências, filtros e o `CHECK` que impede saldo negativo
`test_compras.py`	Recebimento parcial, cancelamento e permissão de quem cria compras
`test_catalogo_clientes.py`	CPF, CNPJ e EAN, conflito de SKU, versão do produto, busca com curinga, documento mascarado e auditoria
`test_relatorios.py`	Fuso de Brasília, vendas, curva ABC, giro e sugestão de compra, painel e CSV
`test_sistema.py`	Saúde, cabeçalhos de segurança, cache da API, arquivos do painel, erro padronizado e coerência do livro
`test_concorrencia.py`	20 confirmações simultâneas para 5 unidades (só em PostgreSQL)

Integração contínua

O workflow `.github/workflows/ci.yml` roda a cada push na `main` e em pull requests:

Job	O que faz
Lint e formatação	`ruff check` e `ruff format --check`
Testes (sqlite)	Suíte completa com cobertura e geração dos dados de demonstração
Testes (postgres)	Suíte completa num PostgreSQL 17, incluindo concorrência, e migrações de ida e volta com `alembic check`
Imagem Docker	Build da imagem depois do lint

Relatório diário automático

Todo dia às 9h17 (horário de Brasília) o workflow `.github/workflows/relatorio-diario.yml` roda os testes, simula um novo dia de operação com uma semente diferente e publica `docs/relatorio-diario.md` com a saúde do código, os indicadores do mês, os pedidos por etapa, os mais vendidos, a reposição sugerida e o histórico dos últimos 30 dias. O robô só altera esse arquivo e faz `git pull --rebase` antes de enviar, então convive com os seus commits.

1. Em **Settings › Actions › General › Workflow permissions**, marque **Read and write permissions**.
2. Opcional: em **Settings › Secrets and variables › Actions › Variables**, crie `GIT_AUTOR_NOME` e `GIT_AUTOR_EMAIL`. Sem elas, os commits saem em nome do `github-actions[bot]`.
3. Para testar na hora: **Actions › Relatório diário › Run workflow**.

Para gerar localmente:

```
pytest -q > resultado-testes.txt
python -m scripts.relatorio_diario
```

Capítulo 12

Dados fictícios e publicação

Dados fictícios

O script `scripts/popular_banco.py` simula meses de operação passando pelas mesmas regras da API, então reservas, baixas e custos são coerentes entre si. São três depósitos (CD-SP em São Paulo, CD-MG em Belo Horizonte e CD-PR em Curitiba), compras recebidas, inventários com avarias e pedidos em todas as situações.

- Nomes de pessoas são combinações aleatórias de prenomes e sobrenomes comuns; empresas têm nomes inventados.
- CPFs e CNPJs são números aleatórios com dígitos verificadores válidos, só para passar na validação.
- E-mails usam os domínios reservados para exemplos (`example.com`, `example.org`, `example.net`).
- Códigos de barras usam o prefixo 2, faixa de circulação restrita que não identifica produtos do varejo.
- As senhas de demonstração são geradas na hora e ficam só no seu computador.

Qualquer semelhança com pessoas ou empresas reais é coincidência.

Publicar o seu fork sem expor dados pessoais

O guia completo está em `docs/c.md`. Em resumo:

1. **Proteja o e-mail dos commits:** no GitHub, em **Settings › Emails**, marque *Keep my email addresses private* e use o endereço `…@users.noreply.github.com` no `git config user.email`.
2. **Confira o que vai ser enviado:** `git status --ignored` deve listar `dados/`, `.venv/` e `.env` como ignorados.
3. **Envie:** crie um repositório vazio no GitHub, faça o primeiro commit e o push. A aba **Actions** mostra o CI rodando.
4. **Opcional:** em **Settings › Code security**, ative *Secret scanning* e *Push protection*.

Capítulo 13

Solução de problemas

Sintoma	O que fazer
`python` não é reconhecido	Instale o Python 3.12+ marcando *Add python.exe to PATH*, ou use o `py` do Windows. O `iniciar.bat` já tenta os dois.
Não sei a senha de demonstração	Abra `dados/credenciais_demo.txt` e `dados/senha_inicial_admin.txt`. Para escolher outras, defina `SENHA_DEMO` e `ADMIN_SENHA` e rode `python -m scripts.popular_banco --limpar`.
A porta 8000 já está em uso	Feche o outro programa ou rode `uvicorn app.main:app --port 8001`. No Docker, mude `PORTA_API`.
"Muitas tentativas" no login	Aguarde 5 minutos. O bloqueio é temporário e vale por origem e por usuário.
Voltei para a tela de entrada sozinho	A sessão expirou (30 minutos) ou foi revogada por troca de senha ou de perfil. Entre de novo.
"Estoque disponível insuficiente"	O detalhe mostra o SKU, o disponível e o solicitado. Reduza a quantidade, troque o depósito ou registre uma entrada.
Desconto recusado	Passou do limite do perfil (vendas 10%, gerência 30%). Peça para a gerência ajustar.
Pedido barrado por crédito	O cliente passou do limite somando os pedidos em aberto. A gerência pode liberar.
HTTP 409 ao salvar	Outra pessoa alterou o registro antes. Recarregue a página e repita a alteração.
Docker para com "variável obrigatória"	Preencha no `.env` a variável citada (`CHAVE_SECRETA`, `POSTGRES_BANCO`, `POSTGRES_USUARIO` ou `POSTGRES_SENHA`).
Não sobe em produção	Com `AMBIENTE=producao`, a `CHAVE_SECRETA` precisa ter 32+ caracteres e `COOKIE_SEGURO=true`.
Quero recomeçar do zero	Pare o servidor e rode `python -m scripts.popular_banco --limpar`, ou apague a pasta `dados/`.
`alembic check` falha	Os modelos mudaram sem migração. Gere uma com `alembic revision --autogenerate`.